

Ejemplo 4.2. La Sociedad para la Prevención de Injusticias con el Atún (SPIA) mantiene una base de datos que registra los votos más recientes de los legisladores en asuntos de importancia para los amantes del atún. Desde el punto de vista conceptual, esta base de datos consta de dos conjuntos de nombres de legisladores llamados *chicos_buenos* y *chicos_malos*. La sociedad olvida con facilidad los errores del pasado y, de igual modo, tiende a olvidar a quienes fueron amigos. Por ejemplo, si se efectuara una votación para decidir acerca de la prohibición de la pesca de atún en el lago Erie, todos los legisladores que votaran a favor se incluirían en *chicos_buenos* y se excluirían de *chicos_malos*, y lo contrario sucedería con los que votaran en contra. Los legisladores que se abstuvieran, permanecerían en el conjunto donde se encontraran, si estuvieran en alguno.

Cuando está en operación, el sistema de bases de datos acepta tres mandatos, cada uno representado por un solo carácter, seguido de una cadena de diez caracteres que denota el nombre de un legislador; cada mandato está en una línea aparte. Los mandatos son:

1. F (sigue un legislador con voto favorable)
2. D (sigue un legislador con voto desfavorable)
3. ? (determina el estado del legislador que sigue).

También se permite el carácter 'E' en la línea de entrada para señalar el fin del proceso. La figura 4.8 muestra un esbozo del programa, escrito en función del TDA DICCIONARIO aún no definido, que en este caso se pretende que sea un conjunto de cadenas de longitud 10. □

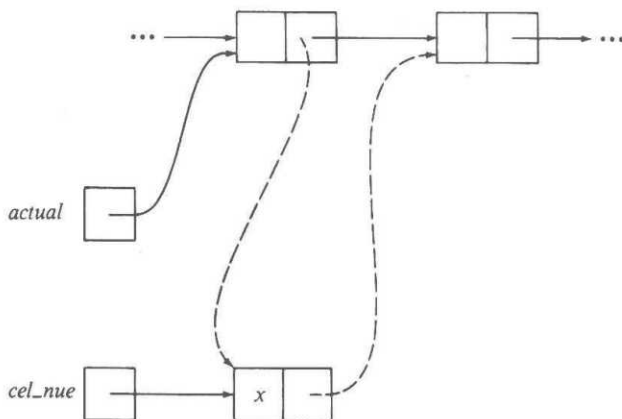


Fig. 4.7. Representación de la inserción.

4.6 Realizaciones sencillas de diccionarios

Un diccionario puede implantarse mediante una lista enlazada, clasificada o no. Otra posible implantación de un diccionario es mediante un vector de bits, siempre que los elementos del conjunto en cuestión estén restringidos a los enteros 1, 2, ..., N para algún N , o a un conjunto que se pueda hacer corresponder con ese conjunto de enteros.

Una posible tercera implantación de diccionarios consiste en usar un arreglo de longitud fija con un apuntador a la última entrada del arreglo en uso. Esta realización sólo es factible si se puede suponer que los conjuntos nunca serán más grandes que la longitud del arreglo. Tiene la ventaja de la sencillez sobre la representación con listas enlazadas, mientras sus desventajas son que 1) los conjuntos no pueden crecer arbitrariamente, 2) la supresión es más lenta, y 3) el espacio no se utiliza en forma eficiente si los conjuntos son de tamaño variable.

A causa de la última razón, no se estudió la realización mediante arreglos en relación con conjuntos cuyas uniones e intersecciones se realizan con frecuencia. No obstante, dado que los arreglos, al igual que las listas, se pueden clasificar, podría considerarse la realización con arreglos que ahora se describe para los diccionarios como una posible realización para conjuntos en general. La figura 4.9 muestra las declaraciones y procedimientos necesarios para complementar la figura 4.8, haciendo de ella un programa ejecutable.

```

program atún ( input, output );
{ base de datos de legisladores }
type
    tipo_nombre = array[1..10] of char;
var
    mandato: char;
    legislador: tipo_nombre;
    chicos_buenos, chicos_malos: DICCIONARIO;

procedure favorable ( amigo: tipo_nombre );
begin
    INSERTA(amigo, chicos_buenos);
    SUPRIME(amigo, chicos_malos)
end; { favorable }

procedure desfavorable ( enemigo: tipo_nombre );
begin
    INSERTA(enemigo, chicos_malos);
    SUPRIME(enemigo, chicos_buenos)
end; { desfavorable }

procedure consulta ( sujeto: tipo_nombre );
begin
    if MIEMBRO(sujeto, chicos_buenos) then
        writeln(sujeto, 'es un amigo')

```

```

    else if MIEMBRO(sujeto, chicos_malos) then
        writeln(sujeto, 'es un enemigo')
    else
        writeln('no se tiene información sobre', sujeto)
    end; { consulta }

begin { programa principal }
    ANULA(chicos_buenos);
    ANULA(chicos_malos);
    read(mandato);
    while mandato <> 'E' do begin
        readln(legislador);
        if mandato = 'F' then
            favorable(legislador)
        else if mandato = 'D' then
            desfavorable(legislador)
        else if mandato = '?' then
            consulta(legislador)
        else
            error('mandato desconocido');
        read(mandato)
    end
end. { aún }

```

Fig. 4.8. Esbozo del programa de base de datos SPIA.

```

const
    tamaño_máx = { algún número adecuado };
type
    DICCIONARIO = record
        últ: integer;
        datos: array[1..tamaño_máx] of tipo_nombre
    end;
procedure ANULA ( var A: DICCIONARIO );
begin
    A.últ := 0
end; { ANULA }
function MIEMBRO ( x: tipo_nombre; var A: DICCIONARIO ) : boolean;
var
    i: integer;
begin
    for i := 1 to A.últ do
        if A.datos[i] = x then return (true);
    return (false) { si no se encuentra x }
end; { MIEMBRO }

```

```

procedure INSERTA ( x: tipo_nombre; var A: DICCIONARIO );
begin
    if not MIEMBRO(x, A) then
        if A.últ < tamaño_máx then begin
            A.últ := A.últ + 1;
            A.datos[A.últ] := x
        end
        else error("la base de datos está llena")
    end; { INSERTA }

procedure SUPRIME ( x: tipo_nombre; var A: DICCIONARIO );
var
    i: integer;
begin
    if A.últ > 0 then begin
        i := 1;
        while (A.datos[i] <> x) and (i < A.últ) do
            i := i + 1;
        { cuando se llega aquí, se ha encontrado x o el elemento
          actual es el último del conjunto A, o ambas cosas }
        if A.datos[i] = x then begin
            A.datos[i] := A.datos[A.últ];
            { pasa el último elemento al lugar ocupado por x; ob-
              sérvese que si i = A.últ, este paso no hace nada,
              pero el siguiente elimina x }
            A.últ := A.últ - 1
        end
    end
end; { SUPRIME }

```

Fig. 4.9. Declaraciones de tipos y procedimientos para un diccionario basado en arreglos.

4.7 La estructura de datos tabla de dispersión

En promedio, la realización de diccionarios mediante arreglos requiere $O(N)$ pasos para la ejecución de una sola instrucción INSERTA, SUPRIME o MIEMBRO en un diccionario de N elementos; si se usa una realización mediante listas, se obtiene una velocidad similar. En la realización mediante vectores de bits, cualquiera de estas tres operaciones utiliza un tiempo constante, pero existe la limitación de trabajar con conjuntos de enteros en un intervalo pequeño para la realización que sea factible.

Existe otra técnica importante y muy útil para implantar diccionarios, y se denomina «dispersión» (*hashing*). La dispersión utiliza tiempo constante por operación, en promedio, y no existe la exigencia de que los conjuntos sean subconjuntos de algún conjunto universal finito. En el peor caso, este método requiere, para

cada operación, un tiempo proporcional al tamaño del conjunto, como sucede con las realizaciones mediante arreglos y listas. Sin embargo, con un diseño cuidadoso es posible hacer que sea arbitrariamente pequeña la probabilidad de que la dispersión demande más de un tiempo constante para cada operación.

Se considerarán dos formas de dispersión algo diferentes. La primera, llamada dispersión *abierta* o *externa*, permite que el conjunto se almacene en un espacio potencialmente ilimitado, por lo que no impone un límite al tamaño del conjunto. La segunda, llamada dispersión *cerrada* o *interna*, usa un espacio fijo para el almacenamiento, por lo que limita el tamaño de los conjuntos †.

Dispersión abierta

En la figura 4.10 se puede ver la estructura de datos básica para la dispersión abierta. La idea fundamental es que el conjunto (posiblemente infinito) de miembros potenciales se divide en un número finito de clases. Si se desea tener B clases, numeradas de 0 a $B-1$, se usa una *función de dispersión* h tal que para cada objeto x del tipo de datos de los miembros del conjunto que se va a representar, $h(x)$ sea uno de los enteros de 0 a $B-1$. Lógicamente, el valor de $h(x)$ es la clase a la cual x pertenece. A menudo se da a x el nombre de *clave* y a $h(x)$ el de *valor de dispersión* de x . A las «clases» se les da el nombre de *cubetas* y se dice que x pertenece a la cubeta $h(x)$.

En un arreglo llamado *tabla de cubetas*, indexado por los *números de cubeta* 0, 1, ..., $B-1$, se tienen los encabezamientos de B listas. Los elementos de la i -ésima lista son los miembros del conjunto que se está representando y que pertenecen a la clase i , esto es, aquellos elementos x del conjunto tales que $h(x) = i$.

Se espera que las cubetas tengan casi el mismo tamaño, de modo que la lista para

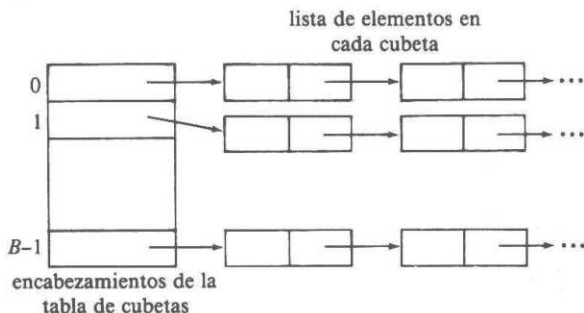


Fig. 4.10. Organización de datos en la dispersión abierta.

† Es posible hacer cambios en la estructura de datos para acelerar la dispersión abierta y permitir que la cerrada maneje conjuntos más grandes. Estas técnicas se describirán después de estudiar los métodos básicos.

cada cubeta sea corta. Entonces, si hay N elementos en el conjunto, en promedio, cada cubeta tendrá N/B miembros. Si se puede estimar N y elegir B aproximadamente igual de grande, entonces una cubeta tendrá en promedio sólo uno o dos miembros, y las operaciones con el diccionario tendrán, en promedio, un número pequeño y constante de pasos, independiente del valor que tenga N (o, en forma equivalente, B).

No siempre está clara la posibilidad de elegir h de modo que un conjunto típico tenga sus miembros distribuidos de forma relativamente uniforme entre las cubetas. Más adelante se estudiará otra vez la forma de elegir h de modo que, en verdad, «disperse» su argumento, es decir, para que $h(x)$ sea un valor «aleatorio» que no dependa de x de ninguna forma trivial.

```
function h ( x: tipo_nombre ) : 0..B-1;
var
  i, suma: integer;
begin
  suma := 0;
  for i := 1 to 10 do
    suma := suma + ord(x[i]);
  h := suma mod B
end; { h }
```

Fig. 4.11. Una función de dispersión sencilla h .

Con el fin de especificar, se presenta ahora una función de dispersión para cadenas de caracteres que es muy buena, aunque no perfecta. La idea es considerar a los caracteres como enteros, utilizando el código de caracteres de la máquina para definir la correspondencia. Pascal proporciona la función incorporada *ord*, donde *ord*(*c*) es el código entero del carácter *c*. Así, si x es una clave y el tipo de las claves es `array[1..10] of char` (lo que se llamó *tipo_nombre* en el Ejemplo 4.2), se podría declarar la función de dispersión h como en la figura 4.11. En esa función, se suman los enteros para cada carácter y se divide el resultado entre B , tomando el residuo, que es un entero entre 0 y $B-1$.

En la figura 4.12 se observan las declaraciones de las estructuras de datos para una tabla de dispersión abierta y los procedimientos que realizan las operaciones para un diccionario. El tipo de los miembros del diccionario se supone que es *tipo_nombre* (un arreglo de caracteres), de modo que estas declaraciones se pueden usar en forma directa en el ejemplo 4.2. Debe observarse que en la figura 4.12 se ha hecho que los encabezamientos de las listas de cubetas sean apuntadores a celdas, en lugar de celdas completas. Esto se hizo así, porque la tabla de cubetas, donde se encuentran los encabezamientos, podría ocupar tanto espacio como las propias listas si fuera un arreglo de celdas, en lugar de uno de apuntadores. Obsérvese, sin embargo, que se paga un precio por este ahorro de espacio. Ahora, el código del procedimiento SUPRIME debe hacer distinción entre la primera celda y las restantes.

```

const
    B = { constante adecuada }
type
    tipo_celda = record
        elemento: tipo_nombre;
        sig: ↑ tipo_celda
    end;
    DICCIONARIO = array[0..B-1] of ↑ tipo_celda;

procedure ANULA ( var A: DICCIONARIO );
var
    i: integer;
begin
    for i := 0 to B-1 do
        A[i] := nil
    end; { ANULA }

function MIEMBRO ( x: tipo_nombre; var A: DICCIONARIO ) : boolean;
var
    actual: ↑ tipo_celda;
begin
    actual := A[h(x)];
    { actual es inicialmente el encabezado de la cubeta para x }
    while actual <> nil do
        if actual↑.elemento = x then
            return (true)
        else
            actual := actual↑.sig;
        return (false) { si no se encuentra x }
    end; { MIEMBRO }

procedure INSERTA ( x: tipo_nombre; var A: DICCIONARIO );
var
    cubeta: integer;
    encab_ant: ↑ tipo_celda;
begin
    if not MIEMBRO(x, A) then begin
        cubeta := h(x);
        encab_ant := A[cubeta];
        new(A[cubeta]);
        A[cubeta]↑.elemento := x;
        A[cubeta]↑.sig := encab_ant
    end
end; { INSERTA }

procedure SUPRIME ( x: tipo_nombre; var A: DICCIONARIO );
var

```

```

actual: ↑ tipo_celda;
cubeta: integer;

begin
  cubeta := h(x);
  if A[cubeta] <> nil then begin
    if A[cubeta].↑.elemento = x then { x está en la primera celda }
      A[cubeta] := A[cubeta].↑.sig { suprime x de la lista }
    else begin { x, si está, no ocupa la primera celda de la cubeta }
      actual := A[cubeta]; { actual apunta a la celda anterior }
      while actual.↑.sig <> nil do
        if actual.↑.sig.↑.elemento = x then begin
          actual.↑.sig := actual.↑.sig.↑.sig
          { suprime x de la lista }
          return { salida del ciclo }
        end
      else { aún no se encuentra x }
        actual := actual.↑.sig
      end
    end
  end
end; { SUPRIME }

```

Fig. 4.12. Realización de un diccionario mediante una tabla de dispersión abierta.

Dispersión cerrada

Una tabla de dispersión cerrada guarda los miembros del diccionario en la tabla de cubetas, en vez de usar esa tabla para almacenar encabezamientos de listas. En consecuencia, parece que sólo es posible colocar un elemento en una cubeta; sin embargo, la dispersión cerrada tiene asociada una *estrategia de redistribución*. Si se intenta colocar x en la cubeta $h(x)$ y esta ya tiene un elemento —situación denominada *colisión*—, la estrategia de redistribución elige una sucesión de localidades alternas $h_1(x)$, $h_2(x)$, ... dentro de la tabla de cubetas, en la cual es posible colocar x . Se prueba en todas esas localidades, en orden, hasta encontrar una vacía. Si ninguna está vacía, la tabla está llena y no es posible insertar x .

Ejemplo 4.3. Supóngase que $B = 8$ y que las claves a , b , c y d tienen valores de dispersión $h(a) = 3$, $h(b) = 0$, $h(c) = 4$, $h(d) = 3$. Se usará la estrategia de redistribución más sencilla, denominada *dispersión lineal*, en la que $h_i(x) = (h(x) + i) \bmod B$. Así, por ejemplo, si se deseara insertar a y se encontrara que la cubeta 3 ya está llena, se probaría con las cubetas 4, 5, 6, 7, 0, 1 y 2, en ese orden.

En principio, se supone que la tabla está vacía, esto es, que cada cubeta guarda un valor especial *vacío*, que no es igual a ningún valor que podría intentarse insertar †. Si se insertan a , b , c y d , en ese orden, en una tabla inicialmente vacía, resulta

† Si el tipo de los miembros del diccionario no sugiere un valor adecuado para *vacío*, se puede hacer que cada cubeta tenga un campo adicional de un bit que informe si está vacía o no.

que a va a la cubeta 3, b a la 0 y c a la 4. Al insertar d , primero se hace la prueba con $h(d) = 3$ para encontrar que está llena. Luego se intenta con $h_1(d) = 4$ y ocurre lo mismo. Por último, se prueba con $h_2(d) = 5$, se encuentra un espacio vacío y d se coloca allí. Las posiciones resultantes están en la figura 4.13. $\square$

La prueba de pertenencia de un elemento x al conjunto requiere examinar $h(x)$, $h_1(x)$, $h_2(x)$, ..., hasta encontrar x o una cubeta vacía. Para ver por qué es posible detenerse al alcanzar una cubeta vacía, supóngase primero que las supresiones no están permitidas. Si $h_3(x)$ es la primera cubeta vacía encontrada en la serie, no es posible que x esté en las cubetas $h_4(x)$, $h_5(x)$ o más adelante en la sucesión, porque x no pudo haber sido colocada allí a menos que $h_3(x)$ hubiera estado llena en el momento de insertarla.

0	b
1	
2	
3	a
4	c
5	d
6	
7	

Fig. 4.13. Tabla de dispersión parcialmente llena.

Sin embargo, obsérvese que si se permiten las supresiones, nunca puede existir la seguridad, si se alcanza una cubeta vacía sin encontrar x , de que x no se encuentra en alguna otra parte, y la cubeta ahora vacía estaba ocupada cuando se insertó x . Cuando deban hacerse supresiones, el enfoque más efectivo para resolver este problema es colocar una constante llamada *suprimido* en una cubeta que contenga un elemento que se desee suprimir. Es importante que haya una diferencia entre *suprimido* y *vacío*, la constante que se encuentra en todas las cubetas que nunca se han llenado. De esta manera, es posible permitir supresiones sin tener que buscar en la tabla completa durante la prueba MIEMBRO. En el momento de la inserción, es posible tratar *suprimido* como un espacio disponible, de modo que con suerte el espacio de un elemento suprimido puede volver a utilizarse. Sin embargo, como el espacio de un elemento suprimido no es reclamable de inmediato, cosa que sí ocurre en la dispersión abierta, puede preferirse el esquema abierto al cerrado.

Ejemplo 4.4. Supóngase que se desea probar si e está en el conjunto representado en la figura 4.13. Si $h(e) = 4$, se prueba con las cubetas 4, 5 y luego 6. La cubeta 6 está vacía y como e no se ha encontrado, la conclusión es que no está en el conjunto.

Si se suprime c , es necesario colocar la constante *suprimido* en la cubeta 4. Así,

al buscar d y comenzar en $h(d) = 3$, se pueden examinar 4 y 5 para encontrar d , y no detenerse en 4 como se hubiera hecho de haber puesto *vacío* en esa cubeta. $\square$

En la figura 4.14 se observan las declaraciones de tipos y las operaciones para el TDA DICCIONARIO con miembros de conjunto del tipo *tipo_nombre* y la tabla de dispersión cerrada como estructura fundamental. Se utiliza una función de dispersión arbitraria h , de la cual la figura 4.11 es una posibilidad, y se usa la estrategia de dispersión lineal para redispersar en caso de colisiones. Por conveniencia, se identifica *vacío* con una cadena de diez espacios y *suprimido* con una de diez asteriscos, con la suposición de que ninguna de esas cadenas puede ser una clave real. (En la base de datos SPIA esas cadenas tienen poca probabilidad de ser nombres de legisladores.) El procedimiento $INSERTA(x, A)$ primero usa *localiza* para determinar si x ya está en A , y si no, utiliza una rutina especial *localiza1* para encontrar una localidad en la cual se pueda insertar x . Obsérvese que *localiza1* busca localidades marcadas tanto con *vacío* como con *suprimido*.

```

const
    vacío = '          '; { 10 espacios }
    suprimido = '*****'; { 10 asteriscos }
type
    DICCIONARIO = array[0..B-1] of tipo_nombre;

procedure ANULA ( var A: DICCIONARIO );
var
    i: integer;
begin
    for i := 0 to B-1 do
        A[i] := vacío;
    end; { ANULA }

function localiza ( x: tipo_nombre ) : integer;
{ localiza examina el DICCIONARIO desde la cubeta para h(x) hasta que
  se encuentre x, o una cubeta vacía, o se haya revisado toda la tabla
  y determinado que no contiene a x; localiza devuelve el índice
  de la cubeta en la que se detiene por cualquiera de esas razones }

var
    inicial, i: integer;
{ inicial guarda h(x); i cuenta el número de cubetas examinadas
  hasta el momento cuando se busca x }
begin
    inicial := h(x);
    i := 0;
    while (i < B) and (A[(inicial + i) mod B] <> x) and
        (A[(inicial + i) mod B] <> vacío) do
        i := i + 1;
    return ((inicial + i) mod B)
end; { localiza }

```

```

function localiza1 ( x: tipo_nombre ): integer;
    { como localiza, pero también se detiene en una entrada con suprimido
      y devuelve ese valor }

function MIEMBRO ( x: tipo_nombre; var A: DICCIONARIO ): boolean;
    begin
        if A[localiza(x)] = x then
            return (true)
        else
            return (false)
        end; { MIEMBRO }

procedure INSERTA ( x: tipo_nombre; var A: DICCIONARIO );
    var
        cubeta: integer;
    begin
        if A[localiza(x)] = x then
            return; { x ya está en A }
            cubeta := localiza1(x);
        if (A[cubeta] = vacío) or (A[cubeta] = suprimido) then
            A[cubeta] := x
        else
            error ('INSERTA falló: la tabla está llena')
        end; { INSERTA }

procedure SUPRIME ( x: tipo_nombre; var A: DICCIONARIO );
    var
        cubeta: integer;
    begin
        cubeta := localiza(x);
        if A[cubeta] = x then
            A[cubeta] := suprimido
        end; { SUPRIME }

```

Fig. 4.14. Realización de un diccionario mediante una tabla de dispersión cerrada.

4.8 Estimación de la eficiencia de las funciones de dispersión

Como ya se mencionó, la dispersión es una manera eficiente de representar diccionarios y otros tipos de datos abstractos basados en conjuntos. En esta sección se examinará el tiempo promedio por operación de diccionario en una tabla de dispersión abierta. Si hay B cubetas y N elementos almacenados en la tabla de dispersión, las cubetas tienen, en promedio, N/B miembros y se puede esperar que una operación normal INSERTA, SUPRIME o MIEMBRO lleve un tiempo $O(1 + N/B)$. La constante 1 representa el tiempo necesario para hallar la cubeta, y N/B , el tiempo para buscar en ella. Si puede elegirse B casi igual a N , este tiempo se convierte

en una constante para cada operación. Por tanto, el tiempo promedio para insertar, suprimir y probar la pertenencia de un elemento al conjunto, suponiendo que un elemento cualquiera tiene la misma probabilidad de ser dispersado en cualquier cubeta, es una constante que no depende de N .

Supóngase que se tiene un programa escrito en algún lenguaje como Pascal y se desea insertar todos los identificadores que aparecen en el programa en una tabla de dispersión. Cada vez que se encuentra la declaración de un nuevo identificador, éste se inserta en la tabla de dispersión después de verificar que ya no se encuentra ahí. Durante esta fase, es razonable suponer que un identificador tiene la misma probabilidad de ser dispersado en cualquier cubeta. Por tanto, se puede construir una tabla de dispersión de N elementos en tiempo $O(N(1 + N/B))$. Al elegir B igual a N , este tiempo es $O(N)$.

En la siguiente fase, los identificadores se hallan en el cuerpo del programa; es necesario localizarlos en la tabla de dispersión para recuperar información asociada con ellos. Pero, ¿cuál es el tiempo estimado para localizar un identificador? Si el elemento que se busca tiene la misma probabilidad que cualquiera de los elementos de la tabla, entonces el tiempo estimado para buscarlo es simplemente el tiempo promedio que demanda la inserción de un elemento. Para comprender esto, obsérvese que el tiempo que demanda buscar una vez cada elemento en la tabla es el empleado en insertarlo, suponiendo que los elementos siempre se agregan al final de la lista de la cubeta apropiada. Entonces, el tiempo estimado para una búsqueda es también $O(1 + N/B)$.

El análisis anterior supone que una función de dispersión distribuye los elementos de manera uniforme en las cubetas. ¿Existen funciones de esta clase? Una función como la de la figura 4.11 (que convierte caracteres en enteros, suma y toma el residuo módulo B) puede considerarse una función típica de dispersión. El siguiente ejemplo examina su funcionamiento.

Ejemplo 4.5. Supóngase que se emplea la función de la figura 4.11 para dispersar 100 claves que consisten en las cadenas de caracteres A0, A1, ..., A99, en una tabla de 100 cubetas. Si se establece que $ord(0)$, $ord(1)$, ..., $ord(9)$ forman una progresión aritmética, como ocurre con los códigos de caracteres más comunes ASCII y EBCDIC, es fácil verificar que las claves se dispersan hasta en 29 de las 100 cubetas † y que la cubeta más grande contiene A18, A27, A36, ..., A90, es decir, 9 de los 100 elementos. Si se calcula el número promedio de pasos para una inserción, partiendo del hecho de que la inserción del i -ésimo elemento en una cubeta requiere $i+1$ pasos, se obtienen 395 pasos para las 100 claves. En comparación, el estimado $N(1+N/B)$ sugiere 200 pasos. □

La función de dispersión sencilla de la figura 4.11 puede tratar ciertos conjuntos de entradas, como las cadenas consecutivas del ejemplo 4.5, de manera no aleatoria. Hay funciones de dispersión «más aleatorias»; como ejemplo, puede usarse la idea de elevar al cuadrado y tomar los dígitos medios. Así, si se tiene como

† Obsérvese que A2 y A20 no necesariamente se dispersan en la misma cubeta, pero, por ejemplo, A23 y A41 deben hacerlo.

clave un número n de 5 dígitos y se eleva al cuadrado, se obtiene un número de 9 ó 10 dígitos. Los «dígitos medios», como los que están entre las posiciones cuarta y séptima a la derecha, tienen valores que dependen de casi todos los dígitos de n ; por ejemplo, el cuarto dígito depende de todos ellos excepto del que se encuentre más a la izquierda de n , y el quinto depende de todos los dígitos de n . De modo que si $B = 100$, se podrían tomar los dígitos sexto y quinto para formar el número de cubeta.

Esta idea se puede generalizar para situaciones en las que B no sea una potencia de 10. Supóngase que las claves son enteros en el intervalo $0, 1, \dots, K$. Si se escoge un entero C tal que BC^2 sea casi igual a K^2 , la función

$$h(n) = \lfloor n^2/C \rfloor \bmod B$$

efectivamente extrae un dígito de base B del centro de n^2 .

Ejemplo 4.6. Si $k = 1000$ y $B = 8$, se puede elegir $C = 354$. Entonces,

$$h(456) = \left\lfloor \frac{207936}{354} \right\rfloor \bmod 8 = 587 \bmod 8 = 3. \quad \square$$

Para usar la estrategia de «elevar al cuadrado y tomar el medio» cuando las claves son cadenas de caracteres, primero se agrupan los caracteres en la cadena de derecha a izquierda, en bloques de un número fijo de caracteres, como 4, rellenando a la izquierda con espacios si es necesario. Se trata cada bloque como un solo entero formado al concatenar los códigos binarios de los caracteres. Por ejemplo, ASCII maneja un código de caracteres de 7 bits, de modo que los caracteres pueden considerarse como «dígitos» de base 2^7 ó 128. De este modo, se puede considerar la cadena de caracteres $abcd$ como el entero $(128)^3 a + (128)^2 b + (128)c + d$. Después de convertir todos los bloques a enteros, se suma $\dagger$ y se procede como se sugirió con anterioridad para los enteros.

Análisis de dispersión cerrada

En un esquema de dispersión cerrada, la velocidad de inserción y de otras operaciones no sólo depende de lo aleatoriamente que la función de dispersión distribuye los elementos en las cubetas, sino también de lo bien que la estrategia de redistribución evita colisiones adicionales cuando una cubeta ya esté llena. Por ejemplo, la estrategia lineal para resolver colisiones no es la mejor posible. Aunque el análisis de este hecho no se efectúa en este libro, es posible observar lo siguiente. Tan pronto como se llenen unas cuantas cubetas consecutivas, cualquier clave que se dispersa a una de ellas será enviada al final del grupo por la estrategia de redistribución, con lo cual el tamaño de ese grupo de cubetas consecutivas se incrementará. De este modo, es probable encontrar sucesiones más largas de cubetas consecutivas

$\dagger$ Si las cadenas pueden ser muy largas, esta suma tendría que hacerse para alguna constante c . Por ejemplo, c valdría uno más que el mayor entero que se pudiera obtener de un solo bloque.

llenas que si los elementos llenaran las cubetas al azar. Más aún, las sucesiones de bloques llenos causan largas secuencias de intentos antes de que un elemento encuentre una cubeta vacía, de modo que tener grupos de cubetas llenas extraordinariamente grandes hace más lentas la inserción y otras operaciones.

Podría desearse saber cuántos intentos (o *sondeos*) se necesitan en promedio para insertar un elemento cuando N de las B cubetas están llenas, al suponer que todas las posibles combinaciones de N de las B cubetas tienen la misma probabilidad de estar llenas. Por lo general se presume, aunque no se demuestre, que ninguna estrategia de dispersión cerrada puede tener un rendimiento temporal medio mejor que ésta para operaciones con diccionarios. Se derivará entonces una fórmula para el costo de inserción cuando las localidades alternas empleadas por la estrategia de redistribución se elijan al azar. Por último, se considerarán algunas estrategias de redistribución, que se aproximan a ese comportamiento aleatorio.

La probabilidad de una colisión en el sondeo inicial es N/B . Suponiendo una colisión, la primera redistribución intentará con una de $B-1$ cubetas, de las cuales $N-1$ estarán llenas, de manera que la probabilidad de al menos dos colisiones es $\frac{N(N-1)}{B(B-1)}$. De modo similar, la probabilidad de al menos i colisiones es

$$\frac{N(N-1) \cdots (N-i+1)}{B(B-1) \cdots (B-i+1)}. \quad (4.3)$$

Si B y N son grandes, esta probabilidad se aproxima a $(N/B)^i$. El número promedio de sondeos es uno (para inserción exitosa) más la suma con todos los $i \geq 1$ de la probabilidad de al menos i colisiones, esto es, alrededor de $1 + \sum_{i=1}^{\infty} (N/B)^i$, o $\frac{B}{B-N}$. Se puede demostrar que el valor exacto de la sumatoria, cuando la fórmula (4.3) se usa para $(N/B)^i$, es $\frac{B+1}{B+1-N}$, de manera que la aproximación obtenida es buena, excepto cuando N se acerca bastante a B .

Obsérvese que $\frac{B+1}{B+1-N}$ crece en una forma muy lenta conforme N comienza a crecer de 0 a $B-1$, que es el mayor valor de N para el cual es posible otra inserción. Por ejemplo, si N es la mitad de B , se necesitan alrededor de dos sondeos para la siguiente inserción. El costo de inserción promedio por cubeta para llenar M de las B cubetas es $\frac{1}{M} \sum_{N=0}^{M-1} \frac{B+1}{B+1-N}$, que se aproxima a $\frac{1}{M} \int_0^{M-1} \frac{B}{B-x} dx$, o $\frac{B}{M} \log_e \left(\frac{B}{B-M+1} \right)$. Llenar por completo la tabla ($M=B$) requiere un promedio de $\log_e B$ sondeos por cubeta o $B \log_e B$ sondeos en total. Sin embargo, llenar la tabla al 90% de su capacidad ($M=0.9B$) sólo requiere $B((10/9) \log_e 10)$ o unos $2.56B$ sondeos.

El costo medio de la prueba de pertenencia para un elemento no existente es idéntico

tico al de insertar el siguiente elemento, pero el costo de la prueba de pertenencia para un elemento que está en el conjunto, es el costo promedio de todas las inserciones realizadas hasta el momento, el cual es menor en gran medida si la tabla está bastante llena. Las supresiones tienen el mismo costo medio que las pruebas de pertenencia, pero, a diferencia de la dispersión abierta, las supresiones de una tabla de dispersión cerrada no ayudan a acelerar las inserciones o pruebas de pertenencia posteriores. Se debe subrayar que si no se permite que las tablas de dispersión cerradas se llenen en más de una fracción fija, menor que uno, de su capacidad total, el costo medio de las operaciones es una constante; esa constante crece conforme lo hace la fracción de la capacidad que se permite usar. En la figura 4.15 se puede ver una gráfica del costo de las inserciones, supresiones y pruebas de pertenencia como una función del porcentaje de la tabla que está lleno cuando se realiza la operación.

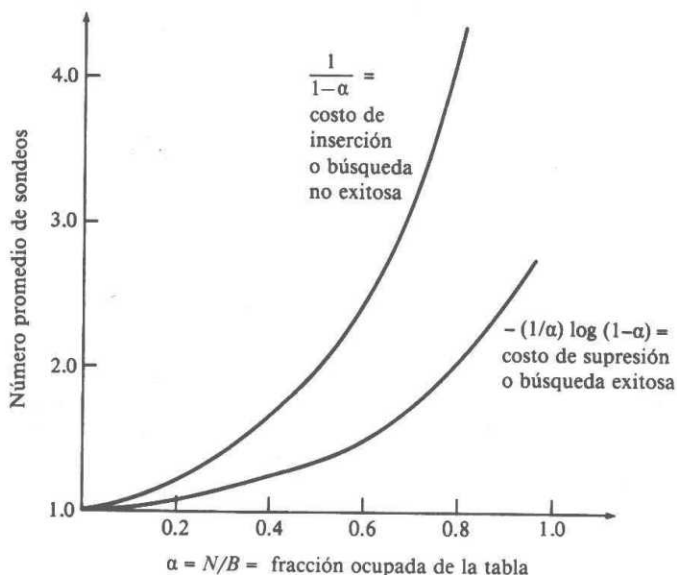


Fig. 4.15. Costo medio de operación.

Estrategias «aleatorias» de resolución de colisiones

Se ha observado que la estrategia de redistribución lineal tiende a agrupar las cubetas llenas en bloques grandes consecutivos. Tal vez se pueda lograr un comportamiento más «aleatorio» si se hacen los sondeos a intervalos constantes mayores que

uno. Esto es, sea $h_i(x) = (h(x) + ci) \bmod B$ para algún $c > 1$. Por ejemplo, si $B = 8$, $c = 3$ y $h(x) = 4$, se sondearían las cubetas 4, 7, 2, 5, 0, 3, 6 y 1, en ese orden. Por supuesto, si c y B tienen un factor común mayor que uno, esta estrategia no permite siquiera buscar en todas las cubetas; como ejemplo, pruébese con $B = 8$ y $c = 2$. Pero todavía más significativo es el hecho de que, aun si c y B son *primos relativos* (no tienen factores comunes), se tiene el mismo problema de «amontonamiento» que con la dispersión lineal, aunque aquí tienden a aparecer las secuencias de cubetas llenas, separadas por una distancia c . Este fenómeno hace más lentas las operaciones, como en la dispersión lineal, puesto que un intento de inserción en una cubeta llena originará un desplazamiento en una cadena de cubetas llenas separadas por la distancia c , y la longitud de esa cadena se incrementará en uno.

En realidad, cualquier estrategia de redistribución en la que el blanco de un sondeo sólo dependa del blanco del sondeo anterior (en lugar de depender del número de sondeos no exitosos ya realizados, la cubeta original $h(n)$, o el valor x de la clave misma) exhibirá la propiedad de amontonamiento de la dispersión lineal. Quizá la estrategia más simple en la que no se presenta este problema sea hacer que $h_i(x) = (h(x) + d_i) \bmod B$, donde $d_1, d_2, \dots, d_{B-1}$, es una permutación «aleatoria» de los enteros $1, 2, \dots, B-1$. Desde luego, en todas las inserciones, supresiones y pruebas de pertenencia se usa la misma sucesión $d_1, \dots, d_{B-1}$; la disposición «aleatoria» de los enteros se decide una sola vez, al diseñar el algoritmo de redistribución.

La generación de números «aleatorios» es un asunto complejo, pero por fortuna muchos métodos comunes producen una sucesión de ellos que van de 1 a cierto límite. Estos generadores de números aleatorios, cuando se reajustan sus valores iniciales para cada operación con la tabla de dispersión, sirven para generar la sucesión deseada $d_1, \dots, d_{B-1}$.

Un enfoque efectivo utiliza «sucesiones de registros de desplazamiento». Sea B una potencia de 2, y k una constante entre 1 y $B-1$. Empiezan con algún número d_1 en el intervalo de 1 a $B-1$, generan los números siguientes de la sucesión tomando el valor anterior, lo doblan y, si el resultado es mayor que B , le sustraen B y toman la suma módulo 2 bit a bit del resultado y la constante seleccionada k . La suma módulo 2 bit a bit de x y y , que se escribe $x \oplus y$, se calcula escribiendo x y y en binario, con ceros a la izquierda si hace falta, para que ambos sean de la misma longitud, y formando los números cuya representación binaria tenga un 1 en las posiciones en que x o y , pero no ambos, también lo tengan.

Ejemplo 4.7. $25 \oplus 13$ se calcula tomando

$$\begin{array}{rcl} 25 & = & 11001 \\ 13 & = & \underline{01101} \\ 25 \oplus 13 & = & 10100 \end{array}$$

Obsérvese que esta «suma» se puede considerar como una suma binaria ordinaria en la que se ignora el residuo. □

No todo valor de k produce una permutación de $1, 2, \dots, B-1$; algunas veces, un número se repite antes de generarlos todos. Sin embargo, para una B dada, existe

una probabilidad pequeña, pero finita, de que algún valor particular de k funcione, y sólo hace falta encontrar una k para cada B .

Ejemplo 4.8. Sea $B = 8$. Si se toma $k = 3$, será fácil generar todos los enteros del 1 al 7. Por ejemplo, si se comienza con $d_1 = 5$, se calcula luego d_2 , doblando primero d_1 para obtener 10. Como $10 > 8$, se resta 8 para obtener 2 y luego se calcula $d_2 = 2 \oplus 3 = 1$. Obsérvese que $x \oplus 3$ se puede calcular al tomar el complemento de los dos últimos bits de x .

Es bueno observar las representaciones binarias en 3 bits de $d_1, d_2, \dots, d_7$ que se muestran en la figura 4.16, junto con su método de cálculo. Obsérvese que la multiplicación por 2, en binario corresponde a un desplazamiento hacia la izquierda. De ahí se obtiene una indicación acerca del origen del término «sucesión de registros de desplazamiento».

Se deberá verificar que también se genera una permutación de 1, 2, ..., 7 si se elige $k = 5$, pero que otros valores de k fracasan. $\square$

	$d_1 = 101 = 5$
desplazamiento	1010
supresión del primer 1	010
$\oplus 3$	$d_2 = 001 = 1$
desplazamiento	$d_3 = 010 = 2$
desplazamiento	$d_4 = 100 = 4$
desplazamiento	1000
supresión del primer 1	000
$\oplus 3$	$d_5 = 011 = 3$
desplazamiento	$d_6 = 110 = 6$
desplazamiento	1100
supresión del primer 1	100
$\oplus 3$	$d_7 = 111 = 7$

Fig. 4.16. Cálculo de una sucesión de registros de desplazamiento.

Reestructuración de tablas de dispersión

Si se usa una tabla de dispersión abierta, el tiempo medio de las operaciones crece con N/B , cantidad que crece con rapidez conforme el número de elementos excede el número de cubetas. De manera similar, para una tabla de dispersión cerrada, como se vio en la figura 4.15, la eficiencia disminuye a medida que N se acerca a B , y no es posible que N exceda a B .

Para mantener el tiempo constante por operación, que en teoría es posible con las tablas de dispersión, se sugiere que si N crece demasiado, por ejemplo $N \geq 0.9 B$ para una tabla cerrada o $N \geq 2 B$ para una abierta, sencillamente se produzca una nueva tabla de dispersión con el doble de cubetas. La inserción de los elementos actuales del conjunto en la nueva tabla llevará, en promedio, menos tiempo que el requerido para insertarlas en la tabla más pequeña, y ese costo se compensa cuando se realizan operaciones de diccionario posteriores.

4.9 Realización del TDA CORRESPONDENCIA

Recuérdese el análisis del TDA CORRESPONDENCIA del capítulo 2, en el cual se definió una correspondencia como una función de los elementos de un dominio a los elementos de un contradominio. Las operaciones de este TDA son:

1. **ANULA**(A) inicializa la correspondencia A , dejando cada elemento del dominio sin valor asignado en el contradominio.
2. **ASIGNA**(A, d, r) define $A(d)$ como r .
3. **CALCULA**(A, d, r) devuelve verdadero y hace que r tome el valor $A(d)$ si $A(d)$ está definido; en caso contrario, devuelve falso.

Las tablas de dispersión son una forma efectiva de obtener correspondencias. Las operaciones **ASIGNA** y **CALCULA** se realizan de la misma manera que las operaciones **INSERTA** y **MIEMBRO** para un diccionario. Considérese primero una tabla de dispersión abierta. Se supone que la función de dispersión $h(d)$ asocia elementos del dominio con números de cubeta. Mientras que para el diccionario las cubetas consistían en listas enlazadas de elementos, para la correspondencia se necesita una lista de elementos del dominio pareados con sus correspondientes valores del contradominio. Para esto, la definición de celda de la figura 4.12 se reemplaza por

```

type
  tipo_celda = record
    elemento_dominio: tipo_dominio;
    contradominio: tipo_contradominio;
    sig: ^tipo_celda
  end

```

donde **tipo_dominio** y **tipo_contradominio** son cualesquiera tipos que tengan los elementos del dominio y contradominio de la correspondencia. La declaración de **CORRESPONDENCIA** queda como

```

type
  CORRESPONDENCIA = array[0..B - 1] of ^tipo_celda

```

Este arreglo es el arreglo de cubetas para una tabla de dispersión. El procedimiento **ASIGNA** está escrito en la figura 4.17. Los códigos de **ANULA** y **CALCULA** se dejan como ejercicio.

De manera parecida, es posible usar una tabla de dispersión cerrada como una correspondencia. Se definen las celdas como campos de elementos del dominio y de elementos del contradominio, y se declara una **CORRESPONDENCIA** como un arreglo de celdas. Como ocurre con las tablas de dispersión abiertas, se hace que la función de dispersión se aplique a los elementos del dominio, no a los del contradominio. Se deja como ejercicio la realización de las operaciones de correspondencia usando una tabla de dispersión cerrada.

4.10 Colas de prioridad

La cola de prioridad es un TDA basado en el modelo de conjunto con las operaciones INSERTA y SUPRIME_MIN, así como con ANULA para asignar valores iniciales a la estructura de datos. Con el fin de definir la nueva operación SUPRIME_MIN, primero se supone que los elementos del conjunto tienen una función «prioridad» definida sobre ellos; para cada elemento a , $p(a)$, la *prioridad* de a es un número real o, en términos más generales, un miembro de algún conjunto linealmente ordenado. La operación INSERTA tiene el significado habitual, mientras que SUPRIME_MIN es una función que devuelve un elemento de más baja prioridad y, como efecto colateral, lo suprime del conjunto. Así, como su nombre indica, SUPRIME_MIN es una combinación de las operaciones SUPRIME y MIN estudiadas antes en este capítulo.

```

procedure ASIGNA ( var A: CORRESPONDENCIA; a: tipo_dominio;
  r: tipo_contradominio );
var
  cubeta: integer;
  actual: ↑ tipo_celda;
begin
  cubeta := h(d);
  actual := A[cubeta];
  while actual < > nil do
    if actual ↑. elemento_dominio = d then begin
      actual ↑. contradominio := r; { reemplaza el valor anterior
        de d }
      return
    end
    else
      actual := actual ↑. sig;
      { en este punto, d no se encontró en la lista }
      actual := A[cubeta]; { usa actual para almacenar la primera celda }
      new(A[cubeta]);
      A[cubeta] ↑. elemento_dominio := d;
      A[cubeta] ↑. contradominio := r;
      A[cubeta] ↑. sig := actual
    end; { ASIGNA }

```

Fig. 4.17. El procedimiento ASIGNA para una tabla de dispersión abierta.

Ejemplo 4.9. El término «cola de prioridad» proviene de la siguiente forma de usar este TDA. La palabra «cola» sugiere la espera de cierto servicio por ciertas personas u objetos, y la palabra «prioridad» sugiere que ese servicio no se proporciona por medio de la disciplina «primero en llegar, primero en ser atendido», que es la base del TDA COLA, sino que cada persona tiene una prioridad basada en la urgencia de su necesidad. Un ejemplo es la sala de espera de un hospital, donde los pacientes

con problemas potencialmente mortales serán atendidos antes que los demás, sin importar el tiempo de espera.

Como ejemplo más corriente del uso de las colas de prioridad, un sistema de cómputo de tiempo compartido necesita mantener un conjunto de procesos que esperan servicio. En general, los diseñadores de estos sistemas desean hacer que los procesos cortos parezcan instantáneos (en la práctica, una respuesta en uno o dos segundos parece instantánea), de modo que a estos procesos se les da prioridad sobre los que ya tienen un consumo sustancial de tiempo. Un proceso que requiera varios segundos de tiempo de cómputo no puede ser instantáneo, por lo que es razonable la estrategia de diferir estos procesos hasta que todos los que tienen posibilidades de parecer instantáneos hayan sido atendidos. Sin embargo, si no se tiene cuidado, los procesos que ya hayan tomado un tiempo mucho mayor que el promedio, quizá no vuelvan a obtener una porción de tiempo y queden esperando para siempre.

Una posible manera de favorecer los procesos cortos sin bloquear los largos, consiste en dar a un proceso P la prioridad $100t_{\text{cons}}(P) - t_{\text{inic}}(P)$. El parámetro t_{cons} es el tiempo consumido hasta el momento por el proceso, y t_{inic} es el momento en que se inició el proceso, medido a partir de cierto «instante cero». Obsérvese que, en general, las prioridades serán enteros negativos grandes, a menos que se elija medir t_{inic} a partir de un instante en el futuro. Obsérvese también que 100 en la fórmula anterior es un «número mágico»; se selecciona de forma que sea algo mayor que el mayor número de procesos que puedan estar activos a la vez. Así, si siempre se escoge el proceso con el número de prioridad más bajo y no hay demasiados procesos cortos, a la larga, un proceso que no termine rápido recibirá el 1% del tiempo de procesador. Si esto es mucho o muy poco, otra constante puede reemplazar el 100 de la fórmula de la prioridad.

Los procesos se representan mediante registros con un identificador de proceso y un número de prioridad. Esto es, se define

```
type
  tipo_proceso = record
    id: integer;
    prioridad: integer
  end;
```

La prioridad de un proceso es el valor del campo de prioridad, que se ha definido como un entero. La función de prioridad se puede definir como sigue:

```
function p (a: tipo_proceso): integer;
begin
  return (a.prioridad)
end;
```

Para seleccionar el proceso que reciba una porción de tiempo, el sistema mantiene una cola de prioridad ESPERA con elementos de tipo tipo_proceso, y emplea dos procedimientos, *inicial* y *selecciona*, para manejar la cola de prioridad mediante las operaciones INSERTA y SUPRIME-MIN. Siempre que un proceso comienza,

se llama al procedimiento *inicial*, el cual coloca un registro para ese proceso en ESPERA. El procedimiento *selecciona* es llamado cuando el sistema dispone de una porción de tiempo asignable a algún proceso. El registro del proceso que resulte seleccionado se elimina de ESPERA, pero *selecciona* lo retiene para reingresarlo en la cola con una nueva prioridad, que es la anterior incrementada en 100 veces la cantidad de tiempo utilizada.

Se emplea la función *tiempo_actual*, que devuelve el tiempo actual en cualesquiera unidades que el sistema use, como microsegundos, y el procedimiento *ejecuta* (*P*) que hace que el proceso con identificador *P* se ejecute durante una porción de tiempo. La figura 4.18 muestra los procedimientos *inicial* y *selecciona*. □

```

procedure inicial ( P: integer );
{ inicial coloca un proceso con identificador P en la cola }
var
    proceso: tipo_proceso;
begin
    proceso.id := P;
    proceso.prioridad := -tiempo_actual;
    INSERTA (proceso, ESPERA)
end; { inicial }

procedure selecciona;
{ selecciona asigna una porción de tiempo al proceso de mayor prioridad }
var
    tiempo_inicio, tiempo_fin: integer;
    proceso: tipo_proceso;
begin
    proceso := ↑ SUPRIME_MIN(ESPERA)
    { SUPRIME_MIN devuelve un apuntador al elemento eliminado }
    tiempo_inicio := tiempo_actual;
    ejecuta(proceso.id);
    tiempo_fin := tiempo_actual;
    proceso.prioridad := proceso.prioridad + 100* (tiempo_fin -
        tiempo_inicio);
    { ajusta la prioridad para incorporar la cantidad de tiempo usado }
    INSERTA (proceso, ESPERA)
    { devuelve el proceso seleccionado a la cola con la nueva prioridad }
end; { selecciona }

```

Fig. 4.18. Asignación de tiempo a procesos.

4.11 Realizaciones de colas de prioridad

Con excepción de la tabla de dispersión, todas las realizaciones estudiadas hasta el momento para los conjuntos son también apropiadas para las colas de prioridad. La razón de que la tabla de dispersión no sea adecuada, es que no proporciona

un medio conveniente de encontrar el menor elemento, por lo que la dispersión sólo agrega complicaciones y no mejora el rendimiento respecto a una lista enlazada, por ejemplo.

En caso de manejarse una lista enlazada, se tienen las opciones de clasificarla o dejarla sin clasificar. Si la lista se clasifica, es fácil encontrar un mínimo; basta tomar su primer elemento. En cambio, la inserción requiere revisar en promedio la mitad de la lista para mantenerla clasificada. Por otro lado, es posible dejar la lista sin clasificar, lo que facilita la inserción y dificulta la selección del mínimo.

Ejemplo 4.10. Se obtendrá SUPRIME_MIN para una lista no clasificada de elementos del tipo tipo_proceso, que se definió en el ejemplo 4.9. El encabezado de la lista es una celda vacía; las aplicaciones de INSERTA y ANULA son directas, y la aplicación por medio de listas clasificadas se deja como ejercicio. La figura 4.19 proporciona la declaración de las celdas, para el tipo COLA_PRIORIDAD, y para el procedimiento SUPRIME_MIN. □

Realización de colas de prioridad mediante árboles parcialmente ordenados

Tanto si se desea emplear listas clasificadas como no clasificadas para representar colas de prioridad, se debe gastar un tiempo proporcional a n para realizar INSERTA o SUPRIME_MIN en conjuntos de tamaño n . Existe otra realización en la que ambas operaciones requieren $O(\log n)$ pasos, una mejora sustancial para valores grandes de n (como $n \geq 100$). La idea básica consiste en organizar los elementos de la cola de prioridad en un árbol binario que esté lo más balanceado posible; hay un ejemplo en la figura 4.20. En el nivel más bajo, en el cual pueden faltar algunas hojas, se exige que las hojas que falten estén a la derecha de las que se encuentran en el nivel más bajo.

Más importante aún, el árbol es *parcialmente ordenado*, es decir, la prioridad del nodo v no es mayor que la de los hijos de v , donde la prioridad de un nodo es el número de prioridad del elemento almacenado en ese nodo. Obsérvese en la figura 4.20 que los nodos con número de prioridad pequeño no necesitan estar a niveles más altos que los de número de prioridad más grande. Por ejemplo, el nivel tres tiene 6 y 8, que son números de prioridad menores que el 9, que aparece en el nivel dos. En cambio, el padre de 6 y 8 tiene prioridad 5, la cual es, y debe ser, al menos tan pequeña como las prioridades de sus hijos.

Para ejecutar SUPRIME_MIN, se devuelve el elemento de menor prioridad, que, como se puede ver con facilidad, debe estar en la raíz. Sin embargo, si lo único que se hace es eliminar la raíz, lo que queda ya no es un árbol. Para que se mantenga la propiedad de árbol parcialmente ordenado, lo más balanceado y con hojas tan a la izquierda posible, se toma la hoja de más a la derecha del nivel más bajo y se coloca de modo provisional en la raíz. La figura 4.21(a) muestra este cambio a partir de la figura 4.20. Después, este elemento se coloca en el árbol lo más abajo posible, intercambiándolo con el hijo que tenga la prioridad más baja, hasta que el elemento

esté en una hoja o en una posición cuya prioridad no sea mayor que la de cualquiera de sus hijos.

En la figura 4.21(a) se debe intercambiar la raíz con su hijo de menor prioridad, el cual tiene prioridad 5. El resultado de este intercambio se muestra en la figura 4.21(b). El elemento que se está desplazando hacia abajo es todavía mayor que sus hijos, los cuales tienen prioridades 6 y 8. Al intercambiarlo con el menor de los dos, 6, se logra el árbol de la figura 4.21(c). Este árbol tiene ya la propiedad de ser parcialmente ordenado.

En esta filtración, si un nodo v tiene un elemento con prioridad a y sus hijos tienen prioridades b y c , y si al menos b o c son menores que a , el intercambio de a con el menor de b y c logra que v tenga un elemento cuya prioridad sea menor que la de cualquiera de sus hijos. Para demostrar esto, supóngase que $b \leq c$. Después del intercambio, el nodo v adquiere la prioridad b , y sus hijos, las prioridades a y c . Se supuso que $b \leq c$ y se dijo que a es mayor que b o c . Por tanto, es seguro que se cumplirá que $b \leq a$. Así, el proceso de inserción esbozado con anterioridad filtrará un elemento hacia abajo en el árbol hasta que no haya más violaciones de la propiedad de orden parcial.

```

type
  tipo_celda = record
    elemento: tipo_proceso;
    siguiente: ↑ tipo_celda
  end;

COLA_CON_PRIORIDAD = ↑ tipo_celda;
  { la celda apuntada es un encabezado de lista }

function SUPRIME_MIN ( var A: COLA_CON_PRIORIDAD ): ↑ tipo_celda;
var
  actual: ↑ tipo_celda; { celda anterior a la "examinada" }
  menor_prioridad: integer; { prioridad más baja encontrada }
  preganador: ↑ tipo_celda; { celda anterior a la del elemento con menor
    prioridad }
begin
  if A ↑ .siguiente = nil then
    error ('no puede encontrar el menor de una lista vacía')
  else begin
    menor_prioridad := p(A ↑ .siguiente ↑ .elemento);
    { p devuelve la prioridad del primer elemento. Obsérvese que
      A apunta a una celda de encabezado que no contiene un
      elemento }
    preganador := A;
    actual := A ↑ .siguiente;
    while actual ↑ .siguiente <> nil do begin
      { compara las prioridades del ganador actual con las del elemento
        siguiente }

```

```

    if  $p(actual \uparrow .siguiente \uparrow .elemento) < menor\_prioridad$  then begin
         $preganador := actual$ ;
         $menor\_prioridad := p(actual \uparrow .siguiente \uparrow .elemento)$ 
    end;
     $actual := actual \uparrow .siguiente$ 
end;
SUPRIME_MIN :=  $preganador \uparrow .siguiente$ ;
    { devuelve el apuntador al ganador }
 $preganador \uparrow .siguiente := preganador \uparrow .siguiente \uparrow .siguiente$ 
    { elimina el ganador de la lista }
end
end; { SUPRIME_MIN }

```

Fig. 4.19. Realización de una cola de prioridad mediante listas enlazadas.

Obsérvese también que SUPRIME_MIN consume un tiempo $O(\log n)$, aplicado a un conjunto de n elementos. Esto ocurre porque ningún camino del árbol tiene más de $1 + \log n$ nodos y el proceso de forzar el descenso de un elemento en el árbol consume un tiempo constante por nodo. Obsérvese que para cualquier constante c , la cantidad $c(1 + \log n)$ es, a lo sumo, igual a $2c \log n$ para $n \geq 2$. Por consiguiente, $c(1 + \log n)$ es $O(\log n)$.

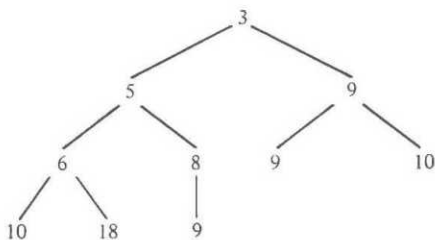


Fig. 4.20. Un árbol parcialmente ordenado.

Ahora se considerará cómo debería trabajar INSERTA. En primer lugar, se coloca el nuevo elemento lo más a la izquierda posible en el nivel más bajo, iniciando un nuevo nivel si el más bajo actual se encuentra lleno en su totalidad. La figura 4.22(a) muestra el resultado de colocar un elemento con prioridad 4 en el árbol de la figura 4.20. Si el nuevo elemento tiene una prioridad más baja que la de su padre, se intercambia con él. El nuevo elemento queda entonces en una posición de menor prioridad que cualquiera de sus hijos, pero puede ser también que tenga menor prioridad que su padre, en cuyo caso se debe intercambiar con él y repetir el proceso hasta que el nuevo elemento llegue hasta la raíz o alcance una posición en la que tenga mayor prioridad que su padre. La figura 4.22(b) y (c) muestra el ascenso de 4 en este proceso.

Se podría demostrar que los pasos anteriores dan como resultado un árbol parcialmente ordenado. No se intentará una demostración rigurosa de este hecho, pero sí se observará que un elemento con prioridad a puede llegar a ser el padre de un elemento con prioridad b de tres maneras. (En el razonamiento siguiente se identificará un elemento con su prioridad.)

1. a es el nuevo elemento y asciende en el árbol reemplazando al padre anterior de b . Sea c la prioridad del padre anterior de b . Entonces, $a < c$, de otro modo, el intercambio no habría tenido lugar. Pero $c \leq b$, puesto que el árbol original estaba parcialmente ordenado. Por tanto, $a < b$. Tómese como ejemplo la figura 4.22(c), donde 4 se convierte en el padre de 6 después de reemplazar a un padre de mayor prioridad, 5.
2. a fue desplazado hacia abajo en el árbol debido a un intercambio con el nuevo elemento. En este caso, a tiene que haber sido un antecesor de b en el árbol parcialmente ordenado original. Por tanto, $a \leq b$. Por ejemplo, en la figura 4.22(c), 5 se convierte en el padre de los elementos con prioridad 8 y 9. Originalmente, 5 era el padre del primero y el «abuelo» del segundo.
3. b sería el nuevo elemento y asciende hasta ser un hijo de a . Si ocurriera que $a > b$, entonces a y b se intercambiarían en el siguiente paso, con lo que se eliminaría la violación de la propiedad parcialmente ordenada.

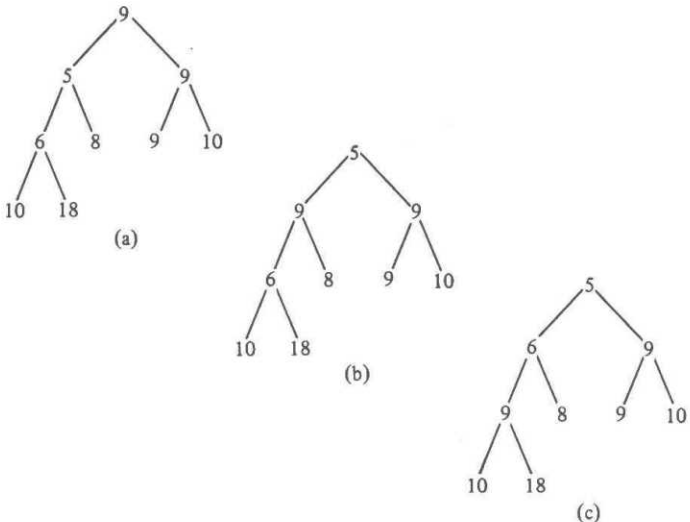


Fig. 4.21. Descenso de un elemento en un árbol.

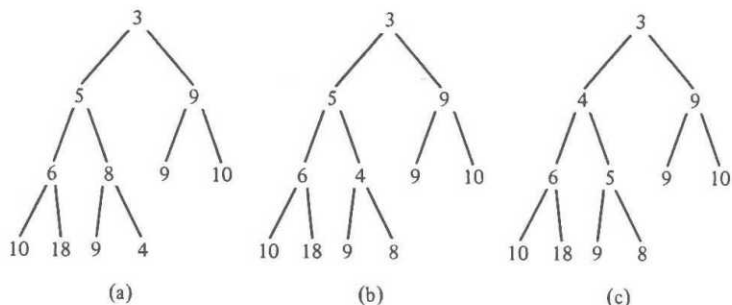


Fig. 4.22. Inserción de un elemento.

El tiempo necesario para efectuar una inserción es proporcional a la distancia que el elemento asciende en el árbol. Como en el caso de SUPRIME_MIN, se observa que esta distancia tal vez no sea mayor que $1 + \log n$, de modo que tanto INSERTA como SUPRIME_MIN dan $O(\log n)$ pasos.

Realización por arreglos de los árboles parcialmente ordenados

El hecho de que los árboles que se han estado considerando sean binarios, lo más balanceados posible, y tengan sus hojas del nivel más bajo lo más a la izquierda posible, permitirá utilizar para ellos una representación poco usual, conocida como *montículo (heap)*. De haber n nodos, se usan las n primeras posiciones de un arreglo A . $A[1]$ contiene la raíz. El hijo izquierdo del nodo $A[i]$, si existe, está en $A[2i]$, y el hijo derecho, si existe, en $A[2i + 1]$. Una manera equivalente de expresar esto es decir que el padre de $A[i]$ es $A[i \text{ div } 2]$, para $i > 1$. Otra observación, también equivalente, es que los nodos del árbol llenan $A[1], A[2], \dots, A[n]$ nivel a nivel, desde arriba, y de izquierda a derecha dentro de cada nivel. Por ejemplo, la figura 4.20 corresponde a un arreglo que contiene los elementos 3, 5, 9, 6, 8, 9, 10, 10, 18, 9.

Se puede declarar una cola de prioridad con elementos de cierto tipo, como tipo_proceso del ejemplo 4.9, como un arreglo de tipos tipo_proceso y un entero *ult* que indica cuál es el último elemento de un arreglo que en un momento dado está en uso. Si se supone que *tam_máx* es el tamaño deseado para los arreglos de colas de prioridad, se puede declarar:

```
type
  COLA_DE_PRIORIDAD = record
    contenido: array[1..tam_máx] of tipo_proceso;
    ult: integer
  end;
```

La realización de las operaciones con colas de prioridad se puede ver en la figura 4.23.

```
procedure ANULA ( var A: COLA_DE_PRIORIDAD );
```

```
begin
```

```
    A.últ := 0
```

```
end; { ANULA }
```

```
procedure INSERTA ( x: tipo_proceso; var A: COLA_DE_PRIORIDAD );
```

```
var
```

```
    i: integer;
```

```
    temp: tipo_proceso;
```

```
begin
```

```
    if A.últ >= long_máx then
```

```
        error('la cola de prioridad está llena')
```

```
    else begin
```

```
        A.últ := A.últ + 1;
```

```
        A.contenido[A.últ] := x;
```

```
        i := A.últ; { i es el índice de la posición actual de x }
```

```
        while (i > 1) and (p(A.contenido[i]) < p(A.contenido[i div 2])) do
```

```
            begin { sube x en el árbol y lo intercambia con su padre  
                    de mayor prioridad. Recuérdese que p calcula  
                    la prioridad de un elemento de tipo_proceso }
```

```
                temp := A.contenido[i];
```

```
                A.contenido[i] := A.contenido[i div 2];
```

```
                A.contenido[i div 2] := temp;
```

```
                i := i div 2
```

```
            end
```

```
        end
```

```
end; { INSERTA }
```

```
function SUPRIME_MIN ( var A: COLA_DE_PRIORIDAD ) : ↑ tipo_proceso;
```

```
var
```

```
    i, j: integer;
```

```
    temp: tipo_proceso;
```

```
    mínimo: ↑ tipo_proceso;
```

```
begin
```

```
    if A.últ = 0 then
```

```
        error('la cola de prioridad está vacía')
```

```
    else begin
```

```
        new(mínimo);
```

```
        mínimo↑ := A.contenido[1];
```

```
        { debe devolverse un apuntador a una copia de la raíz de A }
```

```
        A.contenido[1] := A.contenido[A.últ];
```

```
        A.últ := A.últ - 1;
```

```
        { mueve el último elemento hacia el principio }
```

```
        i := 1; { i es la posición actual del que antes era  
                el último elemento }
```

```

while  $i \leq A.últ \div 2$  do begin
    { lleva hacia abajo el anterior último elemento
      en el árbol }
    if ( $p(A.contenido[2*i]) < p(A.contenido[2*i + 1])$ )
        or ( $2*i = A.últ$ ) then
         $j := 2*i$ 
    else
         $j := 2*i + 1$ ;
    {  $j$  será el hijo de  $i$  que tiene la menor prioridad o bien,
      si  $2*i = A.últ$ ,  $j$  será el único hijo de  $i$  }
    if  $p(A.contenido[i]) > p(A.contenido[j])$  then begin
        { intercambia el anterior último elemento con el hijo
          de menor prioridad }
         $temp := A.contenido[i]$ ;
         $A.contenido[i] := A.contenido[j]$ ;
         $A.contenido[j] := temp$ ;
         $i := j$ 
    end
    else
        return (mínimo) { no puede meterlo más }
    end;
    return (mínimo) { se metió hasta llegar a una hoja }
end
end; { SUPRIME_MIN }

```

Fig. 4.23. Realización de colas de prioridad mediante arreglos.

4.12 Algunas estructuras complejas de conjuntos

En esta sección se considerarán dos usos más complejos de los conjuntos para representar datos. El primer problema será el de representar asociaciones de muchos a muchos, como las que pueden presentarse en un sistema de bases de datos. Un segundo caso de estudio mostrará cómo un par de estructuras de datos que representan el mismo objeto (una correspondencia en el ejemplo que se analizará), pueden dar una representación más eficiente que cualquiera de ellas por separado.

Asociaciones de muchos a muchos y la estructura de listas múltiples

Un ejemplo de asociación de muchos a muchos entre estudiantes y cursos está representada en la figura 4.24. Esta asociación se llama «de muchos a muchos» porque puede haber muchos estudiantes que tomen un curso y cada estudiante puede tomar muchos cursos.

	CS101	CS202	CS303
Alan			X
Alejandro	X	X	
Alicia			X
Amanda	X		
Andrés		X	X
Angélica	X		X

Inscripción

Fig. 4.24. Ejemplo de relación entre estudiantes y cursos.

De vez en cuando, el responsable de los cursos puede desear insertar o eliminar estudiantes de los cursos, determinar cuáles estudiantes están tomando un curso dado, o saber qué cursos está tomando un estudiante dado. La estructura de datos más simple con la que es posible responder estas preguntas es obvia; basta usar el arreglo bidimensional sugerido por la figura 4.24, donde el valor 1 (o verdadero) reemplaza las X y el valor 0 (o falso) reemplaza los espacios.

Por ejemplo, para insertar un estudiante en un curso se necesita una correspondencia, *CE*, tal vez aplicada mediante una tabla de dispersión, que traduzca nombres de estudiante a índices del arreglo y otra, *CC*, que traduzca nombres de curso a índices del arreglo. Entonces, para insertar al estudiante *e* en el curso *c*, simplemente se asigna

$$\text{Inscripción}[CE(e), CC(c)] := 1.$$

Las supresiones se realizan haciendo que este elemento tome el valor 0. Para hallar los cursos que toma un estudiante de nombre *e*, se recorre la fila *CE(e)* y, de manera similar, se recorre la columna *CC(c)* para hallar los estudiantes que están en el curso *c*.

¿Por qué sería deseable buscar una estructura de datos más apropiada? Considérese una universidad grande con, tal vez, 1000 cursos y 20 000 estudiantes, que toman en promedio tres cursos cada uno. El arreglo sugerido por la figura 4.24 tendría 20 000 000 de elementos, de los cuales 60 000, o el 0.3 %, tendrían el valor 1 †. Un arreglo así, que recibe el nombre de *dispersa*, para indicar que casi todos sus elementos valen cero, se puede representar en mucho menos espacio con sólo listar las entradas que no son nulas. Más aún, se puede gastar mucho tiempo revisando una columna de 20 000 entradas en busca de, en promedio, 60 que no son nulos y las revisiones de filas pueden llevar también mucho tiempo.

Una forma de mejorar esto, consiste en plantear el problema como si se tratara del mantenimiento de una colección de conjuntos. Dos de esos conjuntos son *E* y *C*, los conjuntos de todos los estudiantes y todos los cursos. Cada elemento de *E* es en realidad un registro de un tipo como

† Si ésta fuera una base de datos real, el arreglo se guardaría en almacenamiento secundario. Sin embargo, esta estructura de datos malgastaría mucho espacio.

```

type
  tipo_estudiante = record
    ident: integer;
    nombre: array[1..30] of char;
  end

```

y habrá que inventar un tipo de registro parecido para los cursos. Para obtener la estructura pensada, se necesita un tercer conjunto, I , cuyos elementos representen las inscripciones. Cada uno de los elementos de I representará una de las celdas del arreglo de la figura 4.24 que contenga una X. Los elementos de I serían registros de un tipo fijo. Hasta este punto no se sabe qué campos van en esos registros †, pero pronto se sabrá de ellos. Por el momento, basta postular que hay un registro de inscripción por cada entrada marcada con X en la matriz y que los registros de inscripción son distinguibles unos de otros de alguna manera.

También se requieren conjuntos que representen respuestas a las preguntas cruciales: dado un estudiante o un curso, ¿cuáles son los cursos o estudiantes, según el caso, asociados? Sería interesante tener, para cada estudiante e , un conjunto C_e de todos los cursos que e estuviera tomando y, por otro lado, un conjunto E_c de todos los estudiantes que siguen el curso c . Tales conjuntos serían difíciles de obtener, porque no habría límite para el número de conjuntos en los que podría estar cualquier elemento, lo cual daría lugar a complicados registros de estudiantes y cursos. En cambio, sí se podría hacer que E_c y C_e fueran conjuntos de apuntadores, en vez de registros, pero existe un método que permite un ahorro significativo de espacio y ofrece respuestas igual de rápidas a las preguntas sobre estudiantes y cursos.

Sea cada conjunto C_e el conjunto de registros de inscripción correspondientes al estudiante e y algún curso c . Es decir, si se considera una inscripción como un par (e, c) , entonces

$$C_e = \{(e, c) \mid e \text{ está tomando el curso } c\}.$$

De la misma manera, se puede definir

$$E_c = \{(e, c) \mid e \text{ está tomando el curso } c\}.$$

Obsérvese que la única diferencia en el significado de las propiedades de estos dos conjuntos es que, en el primer caso, e es constante, y en el segundo lo es c . Por ejemplo, con base en la figura 4.24, $C_{\text{Alejandro}} = \{(\text{Alejandro}, \text{CS101}), (\text{Alejandro}, \text{CS202})\}$ y $E_{\text{CS101}} = \{(\text{Alejandro}, \text{CS101}), (\text{Amanda}, \text{CS101}), (\text{Angélica}, \text{CS101})\}$.

Estructuras de listas múltiples

En general, una estructura de listas múltiples es cualquier colección de celdas, donde algunas contienen más de un apuntador y pueden, por tanto, pertenecer a más de una lista a la vez. Para cada tipo de celda de una estructura de listas múltiples, es importante distinguir entre los campos apuntadores, de modo que se pueda se-

† En la práctica, sería útil colocar algunos campos, como calificaciones y otros, en los registros de inscripción, pero el planteamiento original del problema no los requiere.

guir una lista en particular sin que haya confusión con respecto a cuál de los diferentes apuntadores de una celda en particular se debe seguir.

Para el caso en cuestión, es posible colocar un campo apuntador en cada registro de estudiante y curso que apunte al primer registro de inscripción en C_e o E_c , respectivamente. Cada registro de inscripción necesita dos campos apuntadores: uno que se llamará *c-sig*, para apuntar a la siguiente inscripción en la lista que representa al conjunto C_e al cual pertenece el registro, y otro, *e-sig*, para apuntar al siguiente elemento del conjunto E_c al que pertenece.

Resulta que un registro de inscripción no indica en forma explícita el estudiante ni el curso que representa. Esa información está implícita en las listas en las cuales aparece el registro de inscripción. Llámese *propietarios* del registro de inscripción a los registros de estudiante y de curso que encabezan estas listas. Entonces, para poder decir qué cursos toma el estudiante e , es preciso examinar los registros de inscripción de C_e y hallar para cada uno su registro de curso propietario. Esto podría hacerse colocando un apuntador en cada registro de inscripción para el registro de curso propietario, y podría necesitarse también un apuntador para el registro de estudiante propietario.

Si bien se pueden usar esos apuntadores con el objeto de responder a las preguntas en el menor tiempo posible, existe la alternativa de lograr un ahorro considerable de espacio †, al costo de hacer más lentos algunos cálculos, si se eliminan los apuntadores y se coloca al final de cada lista E_c un apuntador al registro de curso propietario, y al final de cada lista C_e un apuntador al registro de estudiante propietario. En esta forma, cada registro de estudiante y de curso será parte de un anillo que incluya todos los registros de inscripción de los cuales es propietario. Estos anillos están representados en la figura 4.25, para los datos de la figura 4.24. Obsérvese que los registros de inscripción tienen como primer apuntador *c-sig*, y como segundo, *e-sig*.

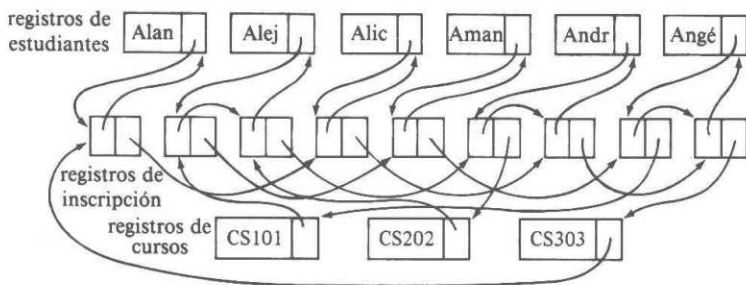


Fig. 4.25. Representación de la figura 4.24 con listas múltiples.

† Obsérvese que es probable que haya muchos más registros de inscripción que registros de estudiantes o cursos, de modo que reducir de tamaño los registros de inscripción permite disminuir la necesidad total de espacio casi en la misma proporción.

Ejemplo 4.11. Para responder a una pregunta como «¿qué estudiantes toman el curso CS101?», se busca el registro de curso correspondiente a CS101. Cómo encontrar este registro depende de la forma en que se mantiene el conjunto de cursos. Por ejemplo, podría haber una tabla de dispersión que tuviera todos esos registros, y para obtener el registro deseado se aplicaría cierta función de dispersión a «CS101».

Luego se sigue el apuntador del registro de CS101 al primer registro de inscripción del anillo de CS101. Este es, en la figura, el segundo registro de inscripción desde la izquierda. Después, hay que buscar al estudiante propietario de este registro de inscripción, lo cual se logra siguiendo los apuntadores *c_sig* (el primero en los registros de inscripción), hasta llegar a un registro de estudiantes †. En este caso, después del tercer registro de inscripción se llega al registro de estudiante para Alejandro; ahora se sabe que Alejandro está tomando el curso CS101.

A continuación, se debe encontrar el siguiente estudiante CS101, y se hace siguiendo el apuntador *e_sig* (el segundo apuntador) del segundo registro de inscripción, el cual conduce al quinto registro de inscripción. El apuntador *c_sig* de ese registro conduce directamente a su propietario, Amanda, de modo que ella está en el curso CS101. Por último, se sigue el apuntador *e_sig* del quinto registro de inscripción hasta el octavo. El anillo de apuntadores *c_sig* de ese registro conduce al noveno registro de inscripción, y de ahí, al registro de estudiante de Angélica, de modo que ella está inscrita en CS101. El apuntador *e_sig* del octavo registro de inscripción conduce de vuelta a CS101, por tanto no hay más estudiantes inscritos en CS101. □

La operación del ejemplo 4.11 se puede expresar en términos abstractos como sigue:

```

for cada registro de inscripción en el conjunto para CS101 do begin
    e := el estudiante propietario del registro de inscripción;
    imprime (e)
end

```

Esta asignación a *e* se puede expresar como

```

f := e;
repeat
    f := f. c_sig
until
    f es un apuntador a un registro de estudiante;
e := campo nombre_estudiante del registro apuntado por f;

```

donde *e* es un apuntador al primer registro de inscripción en el conjunto para CS101.

Para aplicar una estructura como la de la figura 4.25 en Pascal, se necesita un solo tipo de registro, con variantes para cubrir los casos de registros de estudiantes,

† Se debe tener alguna forma de identificar los tipos de registro y se dará momentáneamente una manera de hacerlo.

cursos e inscripciones. En Pascal esto tiene que ser así, ya que los campos *c_sig* y *e_sig* tienen la capacidad de apuntar a tipos de registro distintos. Sin embargo, esta estructura resuelve uno de los problemas que quedan, pues ahora es fácil decir a qué tipo de registro se llega conforme se recorre un anillo. La figura 4.26 muestra una posible declaración de los registros y un procedimiento que imprime los nombres de los estudiantes inscritos en un curso en particular.

Estructuras de datos duales para mayor eficiencia

Con frecuencia, un problema aparentemente simple de representación de un conjunto o correspondencia, conlleva un difícil problema de elección de estructuras de datos. La elección de una estructura de datos para el conjunto simplifica ciertas operaciones, pero hace que otras lleven demasiado tiempo, y, al parecer, no existe una estructura de datos que haga más sencillas todas las operaciones. En tales casos, la solución suele ser el uso simultáneo de dos o más estructuras diferentes para el mismo conjunto o correspondencia.

Supóngase que se desea mantener una «escala de tenistas» en la que cada jugador esté situado en un solo «peldaño». Los jugadores nuevos se agregan en la base de la escala, es decir, en el peldaño con la numeración más alta. Un jugador puede retar a otro que esté en el peldaño inmediato superior, y si le gana, cambia de peldaño con él. Se puede representar esta situación mediante un tipo de datos abstracto cuyo modelo fundamental sea una correspondencia de nombres (cadenas de caracteres) con peldaños (los enteros 1, 2, ...).

Las tres operaciones a realizar son:

1. AGREGA(*nombre*) agrega a la persona nombrada al peldaño de numeración más alta.
2. RETA(*nombre*) es una función que devuelve el nombre de la persona del peldaño $i - 1$ si el jugador nombrado está en el peldaño i , $i > 1$.
3. CAMBIA(i) intercambia los nombres de los jugadores que estén en los peldaños i e $i - 1$, $i > 1$.

type

```

tipo_e = array[1..20] of char;
tipo_c = array[1..5] of char;
clase_registro = ( estudiante, curso, inscripción);
tipo_registro = record
    case clase : clase_registro of
        estudiante : (nombre_estudiante: tipo_e;
                      primer_curso: ↑tipo_registro);
        curso: (nombre_curso: tipo_c;
                primer_estudiante: ↑tipo_registro);
        inscripción: (c_sig, e_sig: ↑tipo_registro)
    end;
end;
```

```

procedure imprime_estudiantes ( nombre_c: tipo_c );
var
  c, e, f:  $\uparrow$ tipo_registro;
begin
  c := apuntador al registro de curso con  $c \uparrow$ .nombre_curso = nombre_c;
  { depende de la aplicación del conjunto curso }
  e :=  $c \uparrow$ .primer_estudiante;
  { e recorre el anillo de inscripciones apuntadas por c }
  while  $e \uparrow$ .clase = inscripción do begin
    f := e;
    repeat
      f :=  $f \uparrow$ .c_sig
    until
       $f \uparrow$ .clase = estudiante;
    { ahora f apunta al estudiante a quien pertenece la inscripción  $e \uparrow$  }

    writeln( $f \uparrow$ .nombre_estudiante);
    e :=  $e \uparrow$ .e_sig
  end
end

```

Fig. 4.26. Realización de una búsqueda en una lista múltiple.

Obsérvese que se ha elegido pasar a CAMBIA sólo el número de peldaño más alto, mientras que las otras dos operaciones tienen un nombre como argumento.

Como alternativa, se podría considerar el uso de un arreglo *ESCALA*, en la que *ESCALA[i]* sea el nombre de la persona del peldaño *i*. Si además se lleva la cuenta del número de jugadores, la adición de un jugador al primer peldaño desocupado se puede hacer en un pequeño número constante de pasos.

CAMBIA también es fácil, puesto que sólo hay que intercambiar dos elementos del arreglo, pero *RETA(nombre)* requiere examinar todo el arreglo en busca del nombre, lo cual lleva un tiempo $O(n)$, si *n* es el número de jugadores en la escala.

Por otra parte, se podría considerar el uso de una tabla de dispersión para representar la correspondencia de nombres a peldaños. En el supuesto de que es posible mantener el número de cubetas proporcional al número de jugadores, *AGREGA* llevaría en promedio un tiempo $O(1)$. A un reto le llevaría un tiempo promedio $O(1)$ buscar el nombre dado, un tiempo $O(n)$ encontrar el nombre que ocupa el peldaño con el siguiente número más bajo, dado que para eso se necesitaría buscar en toda la tabla de dispersión. El intercambio de jugadores requeriría un tiempo $O(n)$ para hallar los jugadores en los peldaños *i* e *i* - 1.

Supóngase, sin embargo, que se combinan las dos estructuras. Las celdas de la tabla de dispersión contendrán pares compuestos de un nombre y un peldaño, mientras que el arreglo tendrá en *ESCALA[i]* un apuntador a la celda correspondiente al jugador que ocupe el peldaño *i*, como se sugiere en la figura 4.27.

- *4.2 Escribese un procedimiento en función de las operaciones básicas con conjuntos que imprima todos los elementos de un conjunto (finito). Puede suponerse que se dispone de un procedimiento para imprimir un objeto del tipo de los elementos. El conjunto a imprimir no debe quedar destruido. ¿Qué estructuras de datos serían las más apropiadas para implantar conjuntos en este caso?
- 4.3 La realización de conjuntos mediante vectores de bits se puede usar siempre que el «conjunto universal» se pueda traducir a los enteros de 1 a N . Describese cómo se haría esa traducción si el conjunto universal fuera
- los enteros 0, 1, ..., 99
 - los enteros de n a m para cualquier $n \leq m$
 - los enteros $n, n+2, n+4, \dots, n+2k$, para cualesquiera n y k
 - los caracteres 'a', 'b', ..., 'z'
 - arreglos de dos caracteres, cada uno de ellos elegidos entre 'a' y 'z'.
- 4.4 Escribanse procedimientos ANULA, UNION, INTERSECCION, MIEMBRO, MIN, INSERTA y SUPRIME para conjuntos representados mediante listas enlazadas, por medio de las operaciones abstractas del TDA lista clasificada. Obsérvese que la figura 4.5 es un procedimiento para INTERSECCION que maneja una realización específica del TDA lista.
- 4.5 Repítase el ejercicio 4.4 para las realizaciones de conjuntos siguientes:
- tabla de dispersión abierta (úsense operaciones abstractas con listas dentro de las cubetas).
 - tabla de dispersión cerrada con resolución lineal de colisiones.
 - lista no clasificada (empléense operaciones abstractas con listas).
 - un arreglo de longitud fija con un apuntador a la última posición usada.
- 4.6 Para cada una de las operaciones y aplicaciones de los ejercicios 4.4 y 4.5, proporciónese el orden de magnitud del tiempo de ejecución con conjuntos de tamaño n .
- 4.7 Supóngase que se están dispersando enteros en una tabla de dispersión de siete cubetas sirviéndose de la función de dispersión $h(i) = i \bmod 7$.
- Muéstrese la tabla de dispersión abierta si se insertan los cubos perfectos 1, 8, 27, 125, 216, 343.
 - Repítase el apartado a) usando una tabla de dispersión cerrada con resolución lineal de colisiones.
- 4.8 Supóngase que se está usando una tabla de dispersión cerrada con cinco cubetas y la función de dispersión $h(i) = i \bmod 5$. Muéstrese la tabla de dispersión cerrada con resolución lineal de colisiones que resulta de insertar la sucesión 23, 48, 35, 4, 10, en una tabla inicialmente vacía.
- 4.9 Obténganse las operaciones del TDA correspondencia, con tablas de dispersión abiertas y cerradas.

- 4.10** Para mejorar la velocidad de las operaciones podría desearse reemplazar una tabla de dispersión abierta con B_1 cubetas con más de B_1 elementos por otra tabla de dispersión con B_2 cubetas. Escribbase un procedimiento para construir la nueva tabla a partir de la anterior, usando las operaciones del TDA lista para procesar cada cubeta.
- 4.11** En la sección 4.8 se habló de las funciones de dispersión «aleatorias» para las cuales $h_i(X)$, la cubeta en la que se va a probar después de i colisiones, es $(h(x) + d_i) \bmod B$ para cierta sucesión $d_1, d_2, \dots, d_{B-1}$. También se sugirió que una manera de calcular una sucesión semejante apropiada era elegir una constante k , y un $d_1 > 0$ arbitrario, y hacer

$$d_i = \begin{cases} 2d_{i-1} & \text{si } 2d_{i-1} < B \\ (2d_{i-1} - B) \oplus k & \text{si } 2d_{i-1} \geq B \end{cases}$$

donde $i > 1$, B es una potencia de 2, y $\oplus$ representa la suma módulo 2 bit a bit. Si $B = 16$, encuentrense los valores de k para los cuales la sucesión $d_1, d_2, \dots, d_{15}$ incluye todos los enteros entre 1 y 15.

- 4.12** a) Muéstrase el árbol parcialmente ordenado que resulta cuando los enteros 5, 6, 4, 9, 3, 1, 7 se insertan en un árbol vacío.
 b) ¿Cuál es el resultado de tres operaciones SUPRIME_MIN sucesivas en el árbol de a)?
- 4.13** Supóngase que se representa el conjunto de cursos mediante
- una lista enlazada
 - una tabla de dispersión
 - un árbol binario de búsqueda.

Modifíquense las declaraciones de la figura 4.26 para cada una de estas estructuras.

- 4.14** Modifíquese la estructura de datos de la figura 4.26 de modo que cada registro de inscripción tenga un apuntador directo al estudiante y curso propietarios. Escribbase otra vez el procedimiento *imprime_estudiantes* de la figura 4.26, aprovechando esta estructura.
- 4.15** Supóngase que hay 20 000 estudiantes, 1000 cursos y cada estudiante en un promedio de tres cursos; compárese la estructura de datos de la figura 4.26 con la modificación sugerida en el ejercicio 4.14 en lo referente a
- la cantidad de espacio requerida
 - el tiempo promedio de ejecución de *imprime_estudiantes*
 - el tiempo promedio de ejecución del procedimiento análogo que imprima los cursos que toma un estudiante dado.
- 4.16** Considérese la estructura de datos de la figura 4.26, dado un registro de curso c y un registro de estudiante e y escribanse procedimientos para insertar y suprimir el hecho de que e toma c .

- 4.17 Si existe, ¿cuál es la diferencia entre la estructura de datos del ejercicio 4.14 y la estructura en la que los conjuntos C_c y E_c se representan mediante listas de apuntadores a los registros de cursos y estudiantes, respectivamente?
- 4.18 Los empleados de cierta compañía se representan en la base de datos de la compañía por su nombre (que se supone único), número de empleado y número de seguridad social. Sugírase una estructura de datos que permita, dada una representación de un empleado, encontrar las otras dos representaciones del mismo individuo. ¿Qué rápida, en promedio, puede lograrse que sea cada una de esas operaciones?

Notas bibliográficas

Knuth [1973] es una buena fuente de información adicional sobre dispersión. La dispersión se desarrolló en la segunda mitad de la década de 1950, y Peterson [1957] es de los primeros artículos fundamentales sobre el tema. Morris [1968] y Maurer y Lewis [1975] dan buena información sobre la dispersión.

La lista múltiple es la estructura de datos central de los sistemas de bases de datos basados en redes propuestos en DBTG [1971]. Ullman [1982] proporciona información adicional sobre las aplicaciones, en bases de datos, de las estructuras de ese tipo.

La aplicación mediante montículos de los árboles parcialmente ordenados se basa en una idea de Williams [1964]. Las colas de prioridad se estudian más a fondo en Knuth [1973].

Reingold [1972] analiza la complejidad computacional de las operaciones básicas con conjuntos. Las técnicas de análisis de flujo de datos basadas en conjuntos se tratan con detalle en Cocke y Allen [1976] y en Aho y Ullman [1977].

5

Métodos avanzados de representación de conjuntos

Este capítulo presenta estructuras de datos para conjuntos que permiten obtener colecciones comunes de operaciones sobre conjuntos más eficientes que las presentadas en el capítulo anterior. Sin embargo, estas estructuras son más complejas y con frecuencia sólo son apropiadas para conjuntos muy grandes. Todas están basadas en varias clases de árboles, como árboles binarios de búsqueda, *tries* (árboles de recuperación de información) y árboles balanceados.

5.1 Árboles binarios de búsqueda

Se comenzará con los árboles binarios de búsqueda, una estructura de datos básica para la representación de conjuntos cuyos elementos están clasificados de acuerdo con algún orden lineal. Como de costumbre, se designará ese orden por medio del signo $<$. Esta estructura de datos es útil cuando se tiene un conjunto de elementos de un universo tan grande, que no es práctico emplear los propios elementos del conjunto como índices de arreglos. Un ejemplo de tal universo puede ser el conjunto de identificadores en un programa en Pascal. Un árbol binario de búsqueda puede manejar las operaciones de conjuntos INSERTA, SUPRIME, MIEMBRO y MIN, tomando en promedio $O(\log n)$ pasos por operación para un conjunto de n elementos.

Un *árbol binario de búsqueda* es un árbol binario en el cual los nodos están etiquetados con elementos de un conjunto. La propiedad importante de este tipo de árboles es que todos los elementos almacenados en el subárbol izquierdo de cualquier nodo x son menores que el elemento almacenado en x , y todos los elementos almacenados en el subárbol derecho de x son mayores que el elemento almacenado en ese sitio. Esta condición, conocida como *propiedad del árbol binario de búsqueda*, se cumple para todo nodo de un árbol binario de búsqueda, incluyendo la raíz.

La figura 5.1 muestra dos árboles binarios de búsqueda que representan el mismo conjunto de enteros. Obsérvese la interesante propiedad de que si se listan los nodos del árbol en orden simétrico, los elementos almacenados en dichos nodos quedan clasificados.

Supóngase que se usa un árbol binario de búsqueda para representar un conjunto. La propiedad del árbol binario de búsqueda hace que sea simple la prueba de pertenencia al conjunto. Para determinar si x es un miembro del conjunto, primero se compara x con el elemento r que se encuentre en la raíz. Si $x = r$, no hay problema, la respuesta a la pregunta de pertenencia es «cierto». Si $x < r$, entonces x , si existe,

sólo puede ser un descendiente del hijo izquierdo de la raíz, a causa de la propiedad del árbol binario de búsqueda †. De igual modo, si $x > r$, x sólo puede ser un descendiente del hijo derecho de la raíz.

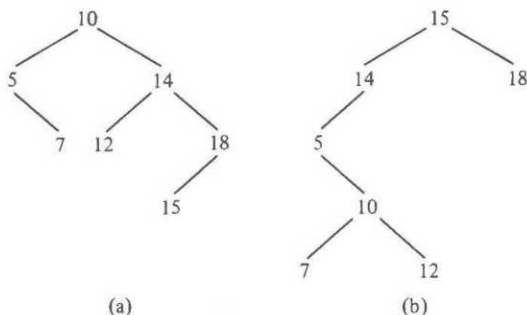


Fig. 5.1. Dos árboles binarios de búsqueda.

Se escribirá una función recursiva simple $\text{MIEMBRO}(x, A)$ para realizar esta prueba de pertenencia. Se supondrá que los elementos del conjunto son de un tipo no especificado que se denominará `tipo_elemento`. Por conveniencia, se supone que `tipo_elemento` es un tipo para el cual están definidos $<$ e $=$. Si no es así, se deben definir las funciones $\text{MQ}(a, b)$ e $\text{IG}(a, b)$, donde a y b son del tipo `tipo_elemento`, tal que $\text{MQ}(a, b)$ es cierto si, y sólo si, a es «menor que» b , e $\text{IG}(a, b)$ es cierto si, y sólo si, a y b son iguales.

El tipo de los nodos consta de un elemento y dos apuntadores a otros nodos:

```

type
  tipo_nodo = record
    elemento: tipo_elemento;
    hijo_izq, hijo_der: † tipo_nodo
  end;

```

Entonces es posible definir el tipo `CONJUNTO` como un apuntador a un nodo, que aquí será la raíz del árbol binario de búsqueda que representa el conjunto. Esto es:

```

type
  CONJUNTO = † tipo_nodo;

```

Ahora se puede especificar en su totalidad la función MIEMBRO de la figura 5.2. Obsérvese que debido a que `CONJUNTO` y «apuntador a `tipo_nodo`» son sinóni-

† Recuérdese que el hijo izquierdo de la raíz es un descendiente de sí mismo, así que no debe eliminarse la posibilidad de que x sea hijo izquierdo de la raíz.

mos, MIEMBRO puede llamarse a sí mismo en subárboles, como si éstos representaran conjuntos. De hecho, el conjunto puede dividirse en el subconjunto de los miembros menores que x y en el subconjunto de los miembros mayores que x .

```

function MIEMBRO (  $x$ : tipo_elemento;  $A$ : CONJUNTO ) : boolean;
{ devuelve verdadero si  $x$  está en  $A$ , y falso en caso contrario }
begin
  if  $A = \text{nil}$  then
    return (false) {  $x$  nunca está en  $\emptyset$  }
  else if  $x = A \uparrow . \text{elemento}$  then
    return (true)
  else if  $x < A \uparrow . \text{elemento}$  then
    return (MIEMBRO( $x$ ,  $A \uparrow . \text{hijo\_izq}$ ))
  else {  $x > A \uparrow . \text{elemento}$  }
    return (MIEMBRO( $x$ ,  $A \uparrow . \text{hijo\_der}$ ))
end; { MIEMBRO }

```

Fig. 5.2. Prueba de pertenencia en un árbol binario de búsqueda.

El procedimiento INSERTA(x , A), que agrega un elemento x al conjunto A , también es fácil de escribir. La primera acción que INSERTA debe efectuar es probar si $A = \text{nil}$, esto es, si el conjunto está vacío. De ser así, se crea un nodo nuevo para colocar x y hacer que A le apunte. Si el conjunto no está vacío, se busca x más o menos como lo hace MIEMBRO, pero al encontrar un apuntador nil durante la búsqueda, se reemplaza por un apuntador a un nodo nuevo que contenga x . Entonces x estará en el lugar correcto, esto es, donde la función MIEMBRO lo encuentre. El código de INSERTA se muestra en la figura 5.3.

```

procedure INSERTA (  $x$ : tipo_elemento; var  $A$ : CONJUNTO );
{ agrega  $x$  al conjunto  $A$  }
begin
  if  $A = \text{nil}$  then begin
    new( $A$ );
     $A \uparrow . \text{elemento} := x$ 
     $A \uparrow . \text{hijo\_izq} := \text{nil}$ ;
     $A \uparrow . \text{hijo\_der} := \text{nil}$ 
  end
  else if  $x < A \uparrow . \text{elemento}$  then
    INSERTA( $x$ ,  $A \uparrow . \text{hijo\_izq}$ )
  else if  $x > A \uparrow . \text{elemento}$  then
    INSERTA( $x$ ,  $A \uparrow . \text{hijo\_der}$ )
  { if  $x = A \uparrow . \text{elemento}$ , no se hace nada;  $x$  ya está en el conjunto }
end; { INSERTA }

```

Fig. 5.3. Inserción de un elemento en un árbol binario de búsqueda.

La eliminación presenta algunos problemas. Primero, se debe localizar el elemento x que se elimine del árbol. Si x está en una hoja, tal vez baste eliminar esa hoja. Sin embargo, x puede estar en un nodo interior *nodo_{-i}*, y eliminar *nodo_{-i}* podría desconectar el árbol.

Si *nodo_{-i}* tiene sólo un hijo, como el nodo 14 de la figura 5.1(b), es posible sustituir *nodo_{-i}* por ese hijo, y el árbol binario de búsqueda quedará bien construido. Si *nodo_{-i}* tiene dos hijos, como el nodo 10 de la figura 5.1(a), es necesario encontrar el menor elemento de los descendientes del hijo derecho †. Por ejemplo, en el caso de que el elemento 10 sea borrado de la figura 5.1(a), se debe reemplazar por 12, el descendiente del hijo derecho de 10 con valor más bajo.

Para escribir SUPRIME, es útil tener una función SUPRIME_MIN(A) que elimine el elemento más pequeño de un árbol no vacío y devuelva el valor del elemento eliminado. El código de SUPRIME_MIN se muestra en la figura 5.4. El código de SUPRIME usa SUPRIME_MIN y se muestra en la figura 5.5.

```
function SUPRIME_MIN ( var A: CONJUNTO ) : tipo_elemento;
    { devuelve y elimina el elemento más pequeño del conjunto A }
begin
    if A↑.hijo_izq = nil then begin
        { A apunta al elemento más pequeño }
        SUPRIME_MIN := A↑.elemento;
        A := A↑.hijo_der;
        { reemplaza el nodo apuntado por A por su hijo derecho }
    end
    else { el nodo apuntado por A tiene un hijo izquierdo }
        SUPRIME_MIN := SUPRIME_MIN(A↑.hijo_izq)
end; { SUPRIME_MIN }
```

Fig. 5.4. Eliminación del elemento más pequeño.

```
procedure SUPRIME ( x: tipo_elemento; var A: CONJUNTO );
    { elimina x del conjunto A }
begin
    if A <> nil then
        if x < A↑.elemento then
            SUPRIME(x, A↑.hijo_izq)
        else if x > A↑.elemento then
            SUPRIME(x, A↑.hijo_der)
        { si se llega aquí, x es el nodo apuntado por A }
        else if (A↑.hijo_izq = nil) and (A↑.hijo_der = nil) then
            A := nil { suprime la hoja que contiene a x }
        else if A↑.hijo_izq = nil then
            A := A↑.hijo_der
```

† Puede ser también el nodo con valor más alto entre los descendientes del hijo izquierdo.

```

else if  $A↑.hijo\_der = \text{nil}$  then
   $A := A↑.hijo\_izq$ 
else { ambos hijos están presentes }
   $A↑.elemento := \text{SUPRIME\_MIN}(A↑.hijo\_der)$ 
end; { SUPRIME }

```

Fig. 5.5. Eliminación en un árbol binario de búsqueda.

Ejemplo 5.1. Supóngase que se intenta eliminar 10 de la figura 5.1(a). Entonces, en la última proposición de SUPRIME se llama a SUPRIME_MIN con un argumento apuntador al nodo 14. Ese apuntador es el campo *hijo_der* de la raíz. Esa llamada produce otra llamada a SUPRIME_MIN. El argumento es ahora un apuntador al nodo 12; este apuntador se encuentra en el campo *hijo_izq* del nodo 14. Se encuentra que 12 no tiene hijo izquierdo, así que se devuelve el elemento 12 y se hace que el hijo izquierdo de 14 sea el hijo derecho de 12, el cual resulta ser nil. Después, SUPRIME toma el valor 12 devuelto por SUPRIME_MIN, que reemplaza a 10. El árbol resultante se muestra en la figura 5.6. □

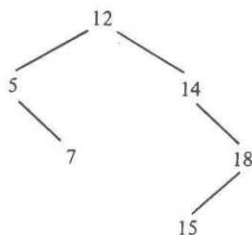


Fig. 5.6. Árbol de la figura 5.1(a) después de suprimir 10.

5.2 Análisis en tiempo de las operaciones para árboles binarios de búsqueda

En esta sección se analiza el comportamiento promedio de distintas operaciones para árboles binarios de búsqueda. Se demuestra que al insertar n elementos aleatorios en un árbol binario de búsqueda inicialmente vacío, la longitud de camino promedio de la raíz a una hoja es $O(\log n)$. La prueba de pertenencia, por tanto, lleva un tiempo $O(\log n)$.

Es fácil ver que si un árbol binario de n nodos está completo (todos los nodos, excepto los que están en el nivel más bajo, tienen dos hijos), ningún camino tendrá más de $1 + \log n$ nodos †. Así, los procedimientos MIEMBRO, INSERTA, SUPRIME y SUPRIME_MIN llevan un tiempo $O(\log n)$. Para comprenderlo, obsérvese que todos toman una cantidad de tiempo constante en un nodo, por lo que pueden llamarse a sí mismos en forma recursiva a lo sumo en un hijo. Por tanto, la secuencia

† Recuérdese que todos los logaritmos son de base 2, a menos que se indique lo contrario.

de nodos en la cual se realizan las llamadas forma un camino desde la raíz. Como el camino es de longitud $O(\log n)$, el tiempo total consumido para seguir el camino es $O(\log n)$.

Sin embargo, al insertar n elementos en un orden «aleatorio», no es forzoso que se acomoden en forma de árbol binario completo. Por ejemplo, si sucede que el primer elemento insertado en orden clasificado es el más pequeño, el árbol resultante será una cadena de n nodos, donde cada nodo, excepto el más bajo en el árbol, tendrá un hijo derecho, pero no un hijo izquierdo. En este caso, es fácil mostrar que como lleva $O(i)$ pasos insertar el i -ésimo elemento y $\sum_{i=1}^n i = n(n+1)/2$, dicho proceso de n inserciones necesita $O(n^2)$ pasos, u $O(n)$ pasos por operación.

Es preciso determinar si el árbol binario de búsqueda «promedio» con n nodos se acerca en estructura al árbol completo y no a la cadena, esto es, si el tiempo promedio por operación en un árbol «aleatorio» necesita $O(\log n)$ pasos, $O(n)$ pasos, o un tiempo intermedio. Como es difícil saber la verdadera frecuencia de inserciones y supresiones, o si los elementos eliminados poseen alguna propiedad especial (por ejemplo, si siempre se elimina el mínimo), sólo se puede analizar la longitud del camino promedio de árboles «aleatorios» adoptando algunas suposiciones: los árboles se forman sólo a partir de inserciones, y todas las magnitudes de los n elementos insertados tienen igual probabilidad.

Con esas suposiciones naturales, se puede calcular $P(n)$, el número promedio de nodos del camino que va de la raíz hacia algún nodo (no necesariamente una hoja). Se supone que el árbol se formó con la inserción de n nodos aleatorios en un árbol que se encontraba vacío en un inicio. Es evidente que $P(0) = 0$ y $P(1) = 1$. Supóngase que se tiene una lista de $n \geq 2$ elementos para insertar en un árbol vacío. El primer elemento en la lista, llamado a , es probable que sea el primero, el segundo o el n -ésimo en el orden de clasificación. Considérese que i elementos en la lista son menores que a , de modo que $n - i - 1$ son mayores que a . Al construir el árbol, a aparecerá en la raíz, los i elementos más pequeños serán descendientes izquierdos de la raíz, y los restantes $n - i - 1$ serán descendientes derechos. (Véase Fig. 5.7.)

Como todos los órdenes de los i elementos pequeños y de los $n - i - 1$ elementos más grandes tienen igual probabilidad, se espera que los subárboles izquierdo y derecho de la raíz tengan longitudes de camino promedio $P(i)$ y $P(n - i - 1)$, respectivamente. Como es posible acceder a esos elementos desde la raíz del árbol comple-

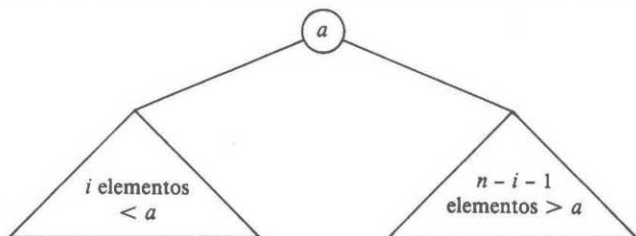


Fig. 5.7. Árbol binario de búsqueda.

to, es necesario agregar 1 al número de nodos de cada camino. Así, para todo i entre 0 y $n-1$, $P(n)$ puede calcularse obteniendo el promedio de la suma

$$\frac{i}{n} (P(i) + 1) + \frac{(n-i-1)}{n} (P(n-i-1) + 1) + \frac{1}{n}$$

El primer término es la longitud de camino promedio en el subárbol izquierdo, ponderando su tamaño. El segundo término es la cantidad análoga del subárbol derecho y el término $1/n$ representa la contribución de la raíz. Al promediar la suma anterior para toda i entre 1 y n , se obtiene la recurrencia

$$P(n) = 1 + \frac{1}{n^2} \sum_{i=0}^{n-1} (iP(i) + (n-i-1) P(n-i-1)) \quad (5.1)$$

La primera parte de la sumatoria (5.1), $\sum_{i=0}^{n-1} iP(i)$, se puede hacer idéntica a la segunda parte $\sum_{i=0}^{n-1} (n-i-1)P(n-i-1)$ si se sustituye i por $n-i-1$ en la segunda parte. Además, el término para $i=0$ del sumatorio $\sum_{i=0}^{n-1} iP(i)$ es cero, por lo que es posible empezar el sumatorio en 1. Así, (5.1) puede escribirse

$$P(n) = 1 + \frac{2}{n^2} \sum_{i=1}^{n-1} iP(i) \quad \text{para } n \geq 2 \quad (5.2)$$

Se demuestra, por inducción sobre n , empezando en $n=1$, que $P(n) \leq 1 + 4\log n$. Con seguridad esta proposición es verdadera para $n=1$, ya que $P(1) = 1$. Supóngase que es verdadera para toda $i < n$. Entonces, por (5.2)

$$\begin{aligned} P(n) &\leq 1 + \frac{2}{n^2} \sum_{i=1}^{n-1} (4i\log i + i) \\ &\leq 1 + \frac{2}{n^2} \sum_{i=1}^{n-1} 4i\log i + \frac{2}{n^2} \sum_{i=1}^{n-1} i \\ &\leq 2 + \frac{8}{n^2} \sum_{i=1}^{n-1} i\log i \end{aligned} \quad (5.3)$$

El último paso está justificado, ya que $\sum_{i=1}^{n-1} i \leq n^2/2$ y, por tanto el último término de la segunda línea es a lo sumo 1. Al dividir los términos del sumatorio de (5.3) en dos partes, aquellos para los que $i \leq [n/2] - 1$, lo cual no excede de $i\log(n/2)$, y aquellos para los que $i > [n/2] - 1$, que no excede de $i\log n$. Así, (5.3) puede escribirse otra vez

$$P(n) \leq 2 + \frac{8}{n^2} \left[\sum_{i=1}^{[n/2]-1} i\log(n/2) + \sum_{i=[n/2]}^{n-1} i\log n \right] \quad (5.4)$$

Sea n par o impar, es posible demostrar que la primera suma de (5.4) no excede de $(n^2/8)\log(n/2)$, lo cual es $(n^2/8)\log n - (n^2/8)$, y la segunda suma no excede de $(3n^2/8)\log n$. Así, (5.4) se puede escribir

$$P(n) \leq 2 + \frac{8}{n^2} \left[\frac{n^2}{2} \log n - \frac{n^2}{8} \right] \\ \leq 1 + 4\log n$$

como se deseaba probar. Este paso completa la inducción y demuestra que el tiempo promedio para seguir un camino de la raíz a un nodo aleatorio de un árbol binario de búsqueda construido mediante inserciones aleatorias es $O(\log n)$, lo cual es, en un factor constante, tan bueno como si el árbol fuera completo. Un análisis más cuidadoso demuestra que la constante 4 es en realidad cercana a 1.4.

Se concluye de lo anterior que el tiempo de la prueba de pertenencia de un miembro aleatorio del conjunto lleva un tiempo $O(\log n)$. Un análisis similar muestra que si se incluyen en la longitud de camino promedio sólo aquellos nodos que carecen de ambos hijos o sólo aquellos que no tienen hijo izquierdo, la longitud del camino promedio aún se ajustará a una ecuación similar a (5.1) y es, por tanto, $O(\log n)$. Es posible aseverar entonces que la prueba de pertenencia de un elemento aleatorio que no está en el conjunto, la inserción de un nuevo elemento aleatorio y la eliminación de un elemento aleatorio también llevan un tiempo $O(\log n)$ en promedio.

Evaluación del rendimiento de los árboles binarios de búsqueda

Las realizaciones de diccionarios por medio de tablas de dispersión requieren un tiempo constante por operación en promedio. Aunque este rendimiento es mejor que el de un árbol binario de búsqueda, una tabla de dispersión requiere $O(n)$ pasos para la operación MIN; así, si MIN se usa con frecuencia, el árbol binario de búsqueda será la mejor opción; si MIN no se usa, tal vez sería preferible la tabla de dispersión.

El árbol binario de búsqueda debe compararse también con el árbol parcialmente ordenado empleado para las colas de prioridad del capítulo 4. Un árbol parcialmente ordenado con n elementos requiere sólo $O(\log n)$ pasos para cada operación INSERTA y SUPRIME_MIN no sólo en el promedio, sino también en el peor caso. Más aún, la constante real de proporcionalidad que acompaña al factor $\log n$ será más pequeña para un árbol parcialmente ordenado que para un árbol binario de búsqueda. Sin embargo, este último permite las operaciones generales SUPRIME y MIN, así como la combinación SUPRIME_MIN, mientras que el árbol parcialmente ordenado sólo permite la última. Además, MIEMBRO requiere $O(n)$ pasos en un árbol parcialmente ordenado, pero sólo $O(\log n)$ pasos en un árbol binario de búsqueda. Así, mientras que el árbol parcialmente ordenado es adecuado para realizar colas de prioridad, no puede efectuar de forma tan eficiente ninguna de las operaciones adicionales que el árbol binario de búsqueda puede hacer.

5.3 Tries

En esta sección se presenta una estructura especial para representar conjuntos de cadenas de caracteres. El mismo método funciona para la representación de tipos de datos que son cadenas de objetos de cualquier tipo, como las cadenas de enteros. Esta estructura se conoce como *trie*, derivada de las letras centrales de la palabra *retrieval* (recuperación) †. A manera de introducción, considérese el siguiente uso de un conjunto de cadenas de caracteres.

Ejemplo 5.2. Como se indicó en el capítulo 1, una forma de implantar un revisor de ortografía es leer un archivo de texto, separarlo en palabras (cadenas de caracteres separados por espacios y caracteres de nueva línea) y encontrar las palabras que no estén en un diccionario estándar de palabras de uso común. Las palabras que estén en el texto, pero no en el diccionario, se imprimen como posibles faltas de ortografía. La figura 5.8 muestra el esbozo de un posible programa *orto*. Este utiliza un procedimiento *toma_palabra(x, t)* que asigna a *x* la siguiente palabra en el archivo de texto *t*; la variable *x* es del tipo llamado *tipo_palabra*, que se define más adelante. La variable *A* es de tipo CONJUNTO; las operaciones necesarias sobre CONJUNTO son INSERTA, SUPRIME, ANULA e IMPRIME. El operador IMPRIME imprime los miembros del conjunto. □

```

program orto ( input, output, diccionario );
  type
    tipo_palabra = { a definir }
    CONJUNTO = { a definir mediante la estructura de trie };
  var
    A: CONJUNTO; { retiene las palabras de entrada no encontradas
                  en el diccionario }
    siguiente_palabra: tipo_palabra;
    diccionario: file of char;

  procedure toma_palabra ( var x: tipo_palabra; f: file of char );
    { procedimiento a definir que hace que x
      sea la siguiente palabra en el archivo f }

  procedure INSERTA ( x: tipo_palabra; var A: CONJUNTO );
    { a definir }

  procedure SUPRIME ( x: tipo_palabra; var A: CONJUNTO );
    { a definir }

  procedure ANULA ( var A: CONJUNTO );
    { a definir }

```

† *Trie* se pensó originalmente como un homónimo de *tree* (que en inglés se pronuncia «tri»), pero para distinguir estos términos, mucha gente prefiere pronunciarlo igual que *pie* (que en inglés se pronuncia «pay»).

```

procedure IMPRIME ( var A: CONJUNTO );
    { a definir }

begin
    ANULA(A);
    while not eof(input) do begin
        toma_palabra(siguiete_palabra, input);
        INSERTA(siguiete_palabra, A)
    end
    while not eof(diccionario) do begin
        toma_palabra(siguiete_palabra, diccionario);
        SUPRIME(siguiete_palabra, A)
    end;
    IMPRIME(A);
end; { orto }

```

Fig. 5.8. Esbozo de revisor de ortografía.

La estructura trie maneja esas operaciones cuando los elementos del conjunto son palabras, esto es, cadenas de caracteres. Es apropiada cuando muchas palabras comienzan con la misma secuencia de letras, es decir, cuando el número de prefijos distintos entre todas las palabras del conjunto es mucho menor que la longitud total de todas las palabras.

En un trie, cada camino de la raíz a una hoja corresponde a una palabra del conjunto representado. De esta forma, los nodos del trie corresponden a los prefijos de las palabras del conjunto. Para evitar confusión entre palabras como ELLO y ELLOS, se añade un símbolo especial *marca_fin*, \$, al final de todas las palabras, y así ningún prefijo de una palabra puede ser una palabra por sí mismo.

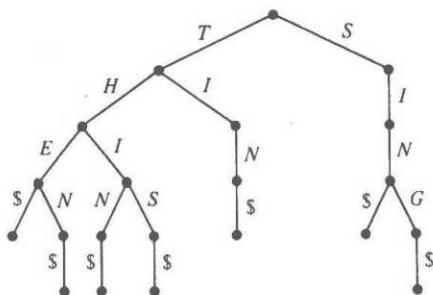


Fig. 5.9. Un trie.

Ejemplo 5.3. En la figura 5.9 hay un trie que representa el conjunto de las palabras {THE, THEN, THIN, THIS, TIN, SIN, SING}. Esto es, la raíz corresponde a la ca-

dena vacía y sus dos hijos corresponden a los prefijos T y S. La hoja que está más a la izquierda representa la palabra THE, la siguiente, la palabra THEN, y así sucesivamente. □

Considérense las siguientes observaciones sobre la figura 5.9.

1. Cada nodo tiene hasta 27 hijos, uno para cada letra y \$.
2. La mayor parte de los nodos tiene mucho menos de 27 hijos.
3. Una hoja que sigue a una arista etiquetada con \$ no puede tener hijos e incluso podría no existir.

Nodos de un trie como TDA

Un nodo de trie puede considerarse como una correspondencia cuyo dominio es $\{A, B, \dots, Z, \$\}$ (o cualquier alfabeto que se escoja) y cuyo conjunto de valores es de tipo «apuntador a nodo de trie». Más aún, el trie mismo puede identificarse con su propia raíz, por lo que los TDA TRIE y NODO_TRIE tienen el mismo tipo de datos, aunque sus operaciones son sustancialmente diferentes. En un NODO_TRIE, se necesitan las operaciones siguientes:

1. procedimiento ASIGNA(*nodo*, *c*, *p*) que asigna el valor *p* (un apuntador a un nodo) al carácter *c* de *nodo*.
2. función VALOR_DE(*nodo*, *c*) que produce el valor asociado con el carácter *c* de *nodo* †, y
3. procedimiento TOMA_NUEVO(*nodo*, *c*) para hacer que el valor de *nodo* para el carácter *c* apunte a un nodo nuevo.

Técnicamente, también se requiere un procedimiento ANULA(*nodo*) para hacer que *nodo* sea la correspondencia nula. Una realización simple de los nodos de un trie es mediante un arreglo *nodo* de apuntadores a nodos, siendo el conjunto de índices $\{A, B, \dots, Z, \$\}$. Esto es, se define

type

```
cars = ('A', 'B', ..., 'Z', '$');
NODO_TRIE = array [cars] of tNODO_TRIE;
```

Si *nodo* es un nodo de trie, *nodo*[*c*] es VALOR_DE(*nodo*, *c*) para cualquier *c* del conjunto de caracteres. Para evitar la creación de muchas hojas que sean hijos correspondientes a '\$', se adoptará la convención de que *nodo*['\$'] es nil o un apuntador al propio nodo. En el primer caso, *nodo* no tiene hijos que correspondan a '\$', y en el segundo caso, se determina que tiene tal hijo, aunque nunca se haya creado. Entonces, se pueden escribir los procedimientos para nodos de trie como en la figura 5.10.

† VALOR_DE es una versión de la función CALCULA de la sección 2.5.

```

procedure ANULA ( var nodo: NODO_TRIE );
{ hace de nodo una hoja, es decir, una correspondencia nula }
var
    c: char;
begin
    for c := 'A' to '$' do
        nodo[c] := nil
    end; { ANULA }

procedure ASIGNA ( var nodo: NODO_TRIE; c: char; p: ↑NODO_TRIE );
begin
    nodo[c] := p
end; { ASIGNA }

function VALOR_DE ( var nodo: NODO_TRIE; c: char ): ↑ NODO_TRIE;
begin
    return (nodo[c])
end; { VALOR_DE }

procedure TOMA_NUEVO ( var nodo: NODO_TRIE; c: char );
begin
    new(nodo[c]);
    ANULA(nodo[c])
end; { TOMA_NUEVO }

```

Fig. 5.10. Operaciones en nodos de un trie.

Ahora, se define

```

type
    TRIE = ↑ NODO_TRIE;

```

Se supondrá que *tipo_palabra* es un arreglo de caracteres de cierta longitud fija. Siempre se partirá del supuesto de que el valor de tal arreglo contiene al menos '\$'; se considerará como fin de la palabra representada el primer '\$', sin importar lo que siga (quizá más '\$'). Con esta suposición, se escribe el procedimiento INSERTA(*x*, *palabras*) para insertar *x* en el conjunto *palabras* representado por un trie, como se muestra en la figura 5.11. Se deja como ejercicio la escritura de ANULA, SUPRIME e IMPRIME para tries representados como arreglos.

```

procedure INSERTA ( x: tipo_palabra; var palabras: TRIE );
var
    i: integer; { cuenta las posiciones en la palabra x }
    t: TRIE; { empleado para apuntar a nodos del trie que corresponden
        a los prefijos de x }

```

```

begin
  i := 1;
  t := palabras;
  while x[i] <> '$' do begin
    if VALOR_DE(t↑, x[i]) = nil then
      { si el nodo actual no tiene hijo para el carácter x[i], crea uno }
      TOMA_NUEVO(t↑, x[i]);
    t := VALOR_DE(t↑, x[i]);
    { prosigue al hijo de t para el carácter x[i], sin importar si
      ese hijo fue creado o no }
    i := i+1 { se mueve en la palabra x }
  end;
  { ahora se ha alcanzado el primer '$' en x }
  ASIGNA(t↑, '$', t)
  { hace un ciclo para '$' para representar una hoja }
end; { INSERTA }

```

Fig. 5.11. Procedimiento INSERTA.

Representación de nodos de un trie por medio de listas

La representación de nodos de un trie mediante arreglos toma una colección de palabras, teniendo en ellas p diferentes prefijos, y las representa con $27p$ bytes de almacenamiento. Esta cantidad de espacio puede exceder con facilidad la longitud total de las palabras del conjunto. Sin embargo, existe otra realización de tries que puede ahorrar espacio. Recuérdese que cada nodo del trie es una correspondencia, como se expuso en la sección 2.6. En principio, cualquier implantación de correspondencias puede funcionar, pero en la práctica se desea una representación adecuada para correspondencias con un dominio pequeño y para las definidas por relativamente pocos miembros del dominio. La representación con listas enlazadas satisface en buena medida esos requisitos. Se puede representar una correspondencia, que es un nodo de un trie, por medio de una lista enlazada de caracteres para la cual el valor asociado no es el apuntador nil. O sea, un nodo de un trie es una lista enlazada de celdas de tipo

```

type
  tipo_celda = record
    dominio: char;
    valor: ↑ tipo_celda;
    {apuntador a la primera celda de la lista para el nodo hijo}
    siguiente: ↑ tipo_celda;
    {apuntador a la siguiente celda de la lista}
  end;

```

Se dejan como ejercicios los procedimientos ASIGNA, VALOR_DE, ANULA y TOMA_NUEVO para esta realización de nodos de trie. Después de escribir esos pro-

cedimientos, las operaciones sobre tries como INSERTA, de la figura 5.11, y otras que quedaron como ejercicios, deben funcionar correctamente.

Evaluación de la estructura de datos trie

Compárense el tiempo y el espacio necesarios para representar n palabras con un total de p prefijos diferentes y una longitud total l usando una tabla de dispersión y un trie. En lo que sigue, se supondrá que los apuntadores requieren cuatro bytes. Quizás el medio más eficaz en cuanto al espacio para almacenar palabras y al manejo de las operaciones INSERTA y SUPRIME sea una tabla de dispersión. Si las palabras son de longitud variable, las celdas de las cubetas no deben contener las palabras mismas, sino que deben constar de dos apuntadores, uno para enlazar entre sí las celdas de la cubeta y otro para apuntar al principio de la palabra que pertenece a la cubeta.

Las palabras mismas se almacenan en un gran arreglo de caracteres, y el fin de cada palabra se indica por medio de un carácter de fin como '\$'. Por ejemplo, las palabras THE, THEN, y THIN pueden almacenarse como

THE\$THEN\$THIN\$...

Los apuntadores de las tres palabras son cursores a las posiciones 1, 5 y 10 del arreglo. La cantidad de espacio utilizado en las cubetas y el arreglo de caracteres es

1. $8n$ bytes para las celdas de las cubetas, siendo una celda para cada una de las n palabras; una celda tiene dos apuntadores u 8 bytes.
2. $l + n$ bytes para que el arreglo de caracteres almacene las n palabras de longitud total l y sus marcas de final.

Así, el espacio total es $9n + l$ bytes más la cantidad empleada para los encabezamientos de las cubetas.

En comparación, un trie con nodos aplicados por medio de listas enlazadas requiere $p + n$ celdas, una celda para cada prefijo y otra para el fin de cada palabra. Cada celda del trie tiene un carácter y dos apuntadores, y necesita nueve bytes, para un espacio total de $9n + 9p$. Si l más el espacio para los encabezados de las cubetas excede de $9p$, el trie usa menos espacio. Sin embargo, para aplicaciones como el almacenamiento de un diccionario en donde l/p es menor que 3, la tabla de dispersión puede ocupar menor espacio.

A favor del trie, sin embargo, obsérvese que se puede recorrer y realizar operaciones tales como INSERTA, SUPRIME y MIEMBRO en un tiempo proporcional a la longitud de la palabra en cuestión. Una función de dispersión que sea realmente «aleatoria» debe comprender cada carácter de la palabra que se esté dispersando. Por tanto, es justo establecer que el cálculo de la función de dispersión lleva tanto tiempo como realizar una operación como MIEMBRO sobre el trie. Por supuesto, el tiempo utilizado en el cálculo de la función de dispersión no incluye el tiempo empleado en resolver las colisiones o la inserción, la eliminación, o la prueba de pertenencia en la tabla de dispersión, así que se puede esperar que los tries sean bas-

tante más rápidos que las tablas de dispersión para diccionarios cuyos elementos son cadenas de caracteres.

Otra ventaja del trie es que permite la realización eficiente de la operación MIN, mientras que las tablas de dispersión, no. Más aún, en la organización con las tablas de dispersión ya descrita, no es posible volver a usar fácilmente el espacio del arreglo de caracteres cuando se borra una palabra (véase en el Cap. 12 los métodos para resolver este problema).

5.4 Realización de conjuntos con árboles balanceados

En las secciones 5.1 y 5.2 se vio cómo realizar conjuntos mediante árboles binarios de búsqueda, y que las operaciones como INSERTA pueden ejecutarse en un tiempo proporcional a la profundidad promedio de los nodos del árbol. Más aún, se hizo patente que esta profundidad promedio es $O(\log n)$ para un árbol «aleatorio» de n nodos. Sin embargo, algunas secuencias de inserciones y eliminaciones pueden producir árboles binarios de búsqueda cuya profundidad promedio sea proporcional a n . Esto sugiere que se puede hacer el intento de reordenar el árbol después de cada inserción y eliminación para que siempre esté completo; entonces el tiempo para las operaciones como INSERTA y similares puede ser siempre $O(\log n)$.

En la figura 5.12(a) se muestra un árbol de seis nodos que se convierte en el árbol completo de 7 nodos mostrado en la figura 5.12(b) cuando se inserta el elemento 1. Todos los elementos de la figura 5.12(a), sin embargo, tienen un padre diferente en la figura 5.12(b), así que deben tomarse n pasos para insertar el 1 en un árbol como el de la figura 5.12(a), si se desea conservar el árbol lo más balanceado posible. Así, es improbable que la sola insistencia en que el árbol binario de búsqueda sea completo lleve a la implantación de un diccionario, cola de prioridad u otro TDA que incluya INSERTA entre sus operaciones, en un tiempo $O(\log n)$.

Existen otros enfoques que dan en el peor caso un tiempo $O(\log n)$ por operación para diccionarios y colas de prioridad, como el llamado «árbol 2-3». Un árbol 2-3 tiene las siguientes propiedades.

1. Cada nodo interior tiene dos o tres hijos.
2. Todos los caminos que van de la raíz a una hoja tienen idéntica longitud.

También se considerará un árbol con uno o con cero nodos, como casos especiales de un árbol 2-3.

Los conjuntos de elementos que están clasificados de acuerdo con algún orden lineal $<$, se representan como sigue. Los elementos están colocados en las hojas; si el elemento a está a la izquierda del elemento b , entonces debe cumplirse que $a < b$. Se supondrá que el ordenamiento « $<$ » de elementos está basado en un campo de un registro que forma el tipo del elemento; este campo se conoce como *clave*. Por ejemplo, los elementos pueden representar personas y cierta información acerca de ellas; en ese caso, el campo clave puede ser el «número de seguridad social».

En cada nodo interior se coloca la clave del elemento más pequeño que sea descendiente del segundo hijo, y si existe un tercer hijo, se coloca también la clave del

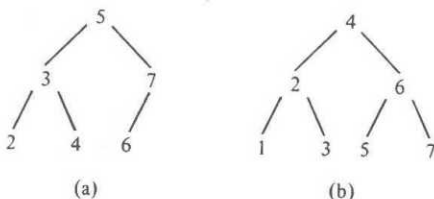


Fig. 5.12. Árboles completos.

elemento más pequeño que descienda de ese hijo †. La figura 5.13 es un ejemplo de árbol 2-3. En ese ejemplo y los siguientes, se identificará un elemento con su campo clave, para que el orden de los elementos sea evidente.

Obsérvese que un árbol 2-3 de k niveles tiene entre 2^{k-1} y 3^{k-1} hojas. Dicho de otra manera, un árbol 2-3 que represente un conjunto de n elementos requiere al menos $1 + \log_3 n$ y no más de $1 + \log_2 n$ niveles. Así, las longitudes de los caminos en el árbol son $O(\log n)$.

Es posible probar la pertenencia de un registro con clave x en un conjunto representado por un árbol 2-3 en un tiempo $O(\log n)$, simplemente descendiendo en el árbol, usando los valores de los elementos registrados en los nodos internos para guiar el camino. En un nodo *nodo*, se compara x con el valor y que representa al menor elemento descendiente del segundo hijo de *nodo*. (Recuérdese que los elementos se están tratando como si consistiesen sólo en un campo clave.) Si $x < y$, hay que ir al primer hijo de *nodo*. Si $x \geq y$ y *nodo* tiene sólo dos hijos, será preciso ir al segundo hijo de *nodo*. Si *nodo* tiene tres hijos y $x \geq y$, debe compararse x con z , el segundo valor registrado en el nodo, el valor que indica el descendiente más pequeño del tercer hijo de *nodo*. Si $x < z$, habrá que ir al segundo hijo, y si $x \geq z$, al tercero. De esta forma, se llega finalmente a una hoja, y x está en el conjunto representado si, y sólo si, x está en la hoja. Es evidente que si durante este proceso $x = y$ o $x = z$,

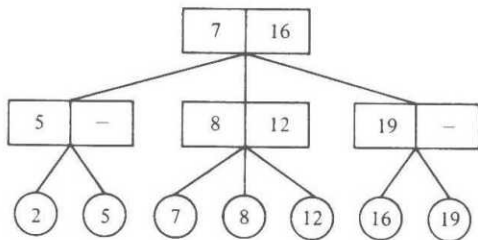


Fig. 5.13. Un árbol 2-3.

† Existe otra versión de árboles 2-3 que coloca registros completos en nodos internos, tal como se hace en los árboles binarios de búsqueda.

se puede parar de inmediato. Sin embargo, el algoritmo quedó así porque en algunos casos es deseable encontrar la hoja con x , además de verificar su existencia.

Inserción en un árbol 2-3

Para insertar un nuevo elemento x en un árbol 2-3, se procede al principio como si se probara la pertenencia de x al conjunto. Sin embargo, justo en el nivel superior al de las hojas, se estará en un nodo *nodo* cuyos hijos no incluyen x . Si *nodo* tiene sólo dos hijos, se hace que x sea el tercero, colocándolo en el orden adecuado. Después se ajustan los dos números de *nodo* para reflejar la nueva situación.

Por ejemplo, al insertar el 18 en la figura 5.13, se termina con *nodo* igual al nodo del extremo derecho en el nivel medio. Se coloca el 18 entre los hijos de *nodo*, cuyo orden correcto es 16, 18, 19. Los dos valores registrados en *nodo* se convierten en 18 y 19, los elementos del segundo y tercer hijos. El resultado se muestra en la figura 5.14.

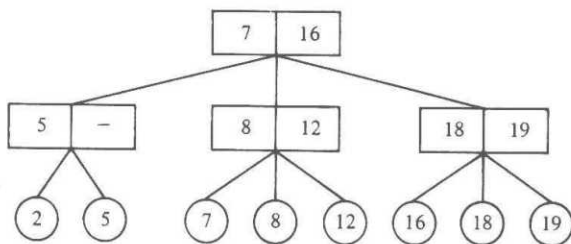


Fig. 5.14. Árbol 2-3 con el 18 insertado.

Sin embargo, supóngase que x es el cuarto hijo de *nodo*, y no el tercero, no es posible tener un nodo con cuatro hijos en un árbol 2-3, por lo que es necesario partir *nodo* en dos: *nodo* y *nodo'*. Los dos elementos más pequeños entre los cuatro hijos de *nodo* permanecen ahí, mientras que los dos más grandes pasan a ser hijos de *nodo'*. Ahora, se debe insertar *nodo'* entre los hijos de p , el padre de *nodo*. Esta parte de la inserción es análoga a la inserción de una hoja como hija de *nodo*. Esto es, si p tiene dos hijos, se hace que *nodo'* sea el tercero y se coloca inmediatamente a la derecha de *nodo*. Si p tiene tres hijos antes de crear *nodo'*, p se divide en p y p' , dejando p para los dos hijos de la izquierda y p' para los dos restantes, y después se inserta p' entre los hijos del padre de p , en forma recursiva.

Un caso especial ocurre cuando se llega a dividir la raíz. En este caso se crea una raíz nueva, cuyos hijos son los dos nodos en los cuales se dividió la raíz primitiva. De esta manera es como se incrementa el número de niveles en un árbol 2-3.

Ejemplo 5.4. Supóngase que se inserta 10 en el árbol de la figura 5.14. El supuesto padre de 10 ya tiene los hijos 7, 8 y 12, así que se divide en dos nodos. El primero

tiene como hijos a 7 y 8, y el segundo, a 10 y 12. El resultado se muestra en la figura 5.15(a). Ahora es necesario insertar en el lugar apropiado un nodo nuevo cuyos hijos sean 10 y 12, como hijo de la raíz de la figura 5.15(a). Al hacerlo, resulta que la raíz tiene cuatro hijos, por lo que se divide, y se crea una raíz nueva, como se muestra en la figura 5.15(b). Los detalles de cómo se lleva hacia arriba la información relacionada con los elementos más pequeños de los subárboles, se darán al desarrollar el programa del mandato INSERTA. □

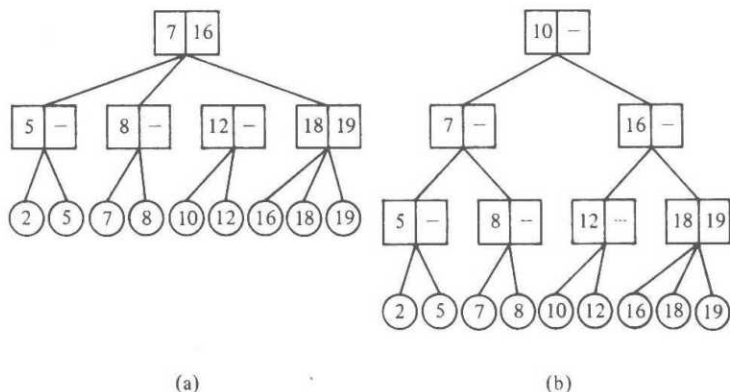


Fig. 5.15. Inserción de 10 en el árbol de la figura 5.14.

Supresión en un árbol 2-3

Al suprimir una hoja, es posible dejar a su *nodo* padre con sólo un hijo. Si *nodo* es la raíz, se suprime *nodo* y se deja el hijo único como nueva raíz. De otra forma, *p* será el padre de *nodo*. Si *p* tiene otro hijo, adyacente a *nodo* por la derecha o por la izquierda, y ese hijo de *p* tiene tres hijos, se puede transferir el más adecuado de esos tres a *nodo*. Entonces *nodo* tendrá dos hijos y se habrá terminado.

Si los hijos de *p* adyacentes a *nodo* tienen sólo dos hijos, se transfiere el único hijo de *nodo* al hermano adyacente de *nodo* y se elimina *nodo*. Si ahora *p* queda sólo con un hijo, se repite todo lo anterior recursivamente, con *p* en lugar de *nodo*.

Ejemplo 5.5. Considérese el árbol de la figura 5.15(b). Si se elimina el 10, su padre tiene sólo un hijo, pero el abuelo tiene otro hijo con tres hijos, 16, 18 y 19. Este nodo está a la derecha del nodo deficitario, por lo que se pasa a ese nodo el elemento más pequeño, 16, quedando el árbol 2-3 de la figura 5.16(a).

A continuación, supóngase que se elimina el 7 del árbol de la figura 5.16(a). Ahora su padre sólo tiene un hijo, 8, y el abuelo no tiene hijos con tres hijos. Por tanto, se hace al 8 hermano de 2 y 5, quedando el árbol de la figura 5.16(b). Ahora el nodo marcado con un asterisco en la figura 5.16(b) tiene sólo un hijo, y su padre no tiene

otros hijos con tres hijos. Se borra entonces el nodo marcado, haciendo que su hijo pase a ser hijo del hermano de ese nodo. Ahora, la raíz tiene sólo un hijo, que se elimina, dejando el árbol de la figura 5.16(c).

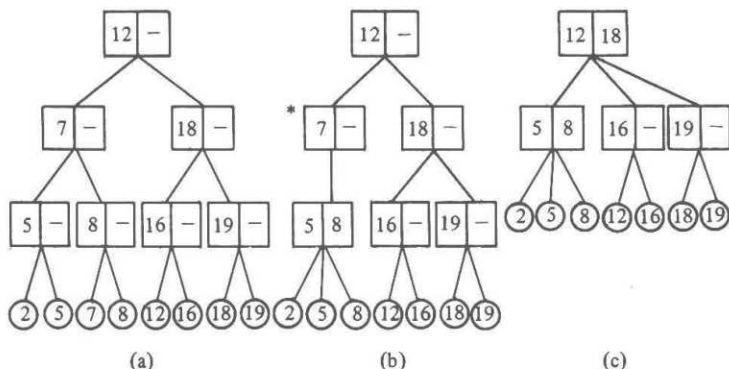


Fig. 5.16. Supresión en un árbol 2-3.

Obsérvese en los ejemplos anteriores la frecuente manipulación de los valores de los nodos interiores. Aunque siempre es posible calcular esos valores recorriendo el árbol, puede hacerse también manipulando el árbol mismo, siempre que se recuerde el valor más pequeño entre los descendientes de cada nodo en el camino que va de la raíz a la hoja eliminada. Esta información puede calcularse con un algoritmo recursivo de eliminación, y con cada llamada en un nodo se pasa, desde arriba, la cantidad correcta (o el valor «menos infinito» si se está en el camino del extremo izquierdo). Los detalles requieren un análisis cuidadoso del caso, y se expondrán al considerar el programa para la operación SUPRIME. □

Tipos de datos para árboles 2-3

Aquí sólo se representarán con árboles 2-3 los conjuntos de elementos cuyas claves sean números reales. La naturaleza de otros campos que van con la clave, para formar un registro de tipo tipo_elemento, se dejará sin especificar, ya que no tiene relación con lo que sigue.

En Pascal, los padres de las hojas deben ser registros que consten de dos números reales (las claves de los elementos más pequeños en el segundo y tercer subárbol) y de tres apuntadores a elementos. Los padres de esos nodos son registros que comprenden dos números reales y tres apuntadores a padres de hojas. Esta progresión continúa indefinidamente: cada nivel de un árbol 2-3 es de un tipo diferente al de los otros niveles. Esta situación haría imposible programar en Pascal las operaciones de árboles 2-3, pero por fortuna, Pascal ofrece un mecanismo, la estructura

de registro variante, que permite considerar a todos los nodos del árbol 2-3 como del mismo tipo, aun cuando algunos sean elementos y otros sean registros con apuntadores y números reales †. Se pueden definir los nodos como en la figura 5.17. Entonces se declararía un conjunto, representado mediante un árbol 2-3, como un apuntador a la raíz, como se muestra en la figura 5.17.

```

type
  tipo_elemento = record
    clave: real;
    { los demás campos requeridos }
  end;
  tipos_nodo = (hoja, interior);
  nodo_dos_tres = record
    case clase: tipos_nodo of
      hoja : (elemento: tipo_elemento);
      interior: (primer_hijo, segundo_hijo, tercer_hijo: † nodo_dos_tres;
        menor_de_segundo, menor_de_tercero: real)
    end;
  CONJUNTO = † nodo_dos_tres;

```

Fig. 5.17. Definición de un nodo en un árbol 2-3.

Realización de INSERTA

Los detalles de las operaciones en árboles 2-3 son muy complicados, aunque los principios son simples. Así pues, se describe con detalle sólo una operación, la inserción; las otras, supresión y prueba de pertenencia, son similares en esencia, y encontrar el mínimo requiere una búsqueda trivial en el camino del extremo izquierdo. Se escribirá la rutina de inserción como procedimiento principal, INSERTA, que se llama en la raíz, y un procedimiento *inserta1*, al cual se llama recursivamente en el árbol. Por conveniencia, se supondrá que un árbol 2-3 no es un árbol vacío o con un solo nodo. Esos dos casos requieren una secuencia directa de pasos que se recomiendan obtener como ejercicio.

Se desea que *inserta1* devuelva un apuntador a un nuevo nodo, si ha de crearlo, y la clave del elemento más pequeño que descende del nuevo nodo. Como el mecanismo de Pascal para crear tal función es complejo, se declarará *inserta1* como un procedimiento que asigna valores a los parámetros *ap_nuevo* y *menor* en caso que deba «devolver» un nodo nuevo. En la figura 5.18 se muestra un esbozo de *inserta1*. El procedimiento completo se muestra en la figura 5.19; para ahorrar espacio, en la 5.19 se han omitido algunos comentarios de la figura 5.18.

† Sin embargo, todos los nodos toman la mayor cantidad de espacio necesaria para cualquiera de los tipos variantes, así que Pascal no es en realidad el mejor lenguaje para la realización práctica de árboles 2-3.

```

procedure insertal ( nodo: ↑ nodo_dos_tres;
  x: tipo_elemento; { x se insertará en el subárbol de nodo }
  var ap_nuevo: ↑ nodo_dos_tres; { apuntador al nodo recién creado
    a la derecha de nodo }
  var menor: real ); { elemento más pequeño del subárbol al que
    apunta ap_nuevo }

begin
  ap_nuevo := nil;
  if nodo es una hoja then begin
    if x no es el elemento que está en nodo then begin
      crea un nodo nuevo apuntado por ap_nuevo;
      pone x en el nodo nuevo;
      menor := x.llave
    end
  end
  else begin { nodo es un nodo interno }
    sea w el hijo de nodo a cuyo subárbol pertenece x;
    insertal(w, x, ap_atrás, menor_atrás);
    if ap_atrás <> nil then begin
      inserta el apuntador ap_atrás entre los hijos de
        nodo justo a la derecha de w;
      if nodo tiene cuatro hijos then begin
        crea un nodo nuevo apuntado por ap_nuevo;
        da al nuevo nodo los hijos tercero y cuarto de nodo;
        ajusta menor_de_segundo y menor_de_tercero en nodo
          y el nodo nuevo;
        coloca menor como la menor clave entre los hijos
          del nodo nuevo
      end
    end
  end
end; { insertal }

```

Fig. 5.18. Esbozo del programa para inserción en árboles 2-3.

```

procedure insertal ( nodo: ↑ nodo_dos_tres; x: tipo_elemento;
  var ap_nuevo: ↑ nodo_dos_tres; var menor: real );

var
  ap_atrás: ↑ nodo_dos_tres;
  menor_atrás: real;
  hijo: 1..3 ; { indica qué hijo de nodo se sigue en la llamada
    recursiva (véase w en la Fig. 5.18) }
  w: ↑ nodo_dos_tres; { apuntador al hijo }

```

```

begin
  ap_nuevo := nil;
  if nodo↑.clase = hoja then begin
    if nodo↑.elemento.clave <> x.clave then begin
      { crea una hoja nueva que contiene x.clave y "devuelve"
        este nodo }
      new(ap_nuevo, hoja);
      if (nodo↑.elemento.clave < x.clave) then
        { coloca x en el nuevo nodo a la derecha del nodo actual }
        begin ap_nuevo↑.elemento := x; menor := x.clave end
      else begin { x está a la izquierda del elemento en el
        nodo actual }
        ap_nuevo↑.elemento := nodo↑.elemento;
        nodo↑.elemento := x;
        menor := ap_nuevo↑.elemento.clave
      end
    end
  end
else begin { nodo es un nodo interno }
  { selecciona el hijo de nodo que se debe seguir }
  if x.clave < nodo↑.menor_de_segundo then
    begin hijo := 1; w := nodo↑.primer_hijo end
  else if (nodo↑.tercer_hijo = nil) or (x.clave < nodo↑.menor_de_tercero)
  then begin
    { x está en el segundo subárbol }
    hijo := 2;
    w := nodo↑.segundo_hijo
  end
  else begin { x está en el tercer subárbol }
    hijo := 3;
    w := nodo↑.tercer_hijo
  end;
  inserta1(w, x, ap_atrás, menor_atrás);
  if ap_atrás <> nil then
    { debe insertarse un nuevo hijo de nodo }
    if nodo↑.tercer_hijo = nil then
      { nodo tiene sólo dos hijos, así que se inserta el nuevo en el
        lugar adecuado }
      if hijo = 2 then begin
        nodo↑.tercer_hijo := ap_atrás;
        nodo↑.menor_de_tercero := menor_atrás
      end
      else begin { hijo = 1 }
        nodo↑.tercer_hijo := nodo↑.segundo_hijo;
        nodo↑.menor_de_tercero := nodo↑.menor_de_segundo;
        nodo↑.segundo_hijo := ap_atrás;

```